

YOLO11-CARD-CPP

Generated by Doxygen 1.14.0

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Shared		??
------------------------	-------	----

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Detection		??
Detections		??
MetricsCounter		??
TerminalRawMode		??
UserData		??
YOLO_model		??

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

include/ hermann_serain.hpp	??
include/ marco_annunziata.hpp	??
include/ shared.hpp	??
include/ sveva_turola.hpp	??
src/ finalProject.cpp	??
src/ hermann_serain.cpp	??
src/ marco_annunziata.cpp	??
src/ shared.cpp	??
src/ sveva_turola.cpp	??

Chapter 4

Namespace Documentation

4.1 Shared Namespace Reference

Enumerations

- enum class [Choice](#) { [Camera](#) , [File](#) , [Invalid](#) , [Help](#) }

Functions

- [Choice](#) [parseChoice](#) (std::string s)

4.1.1 Enumeration Type Documentation

4.1.1.1 Choice

```
enum class Shared::Choice [strong]
```

Enumerator

Camera	
File	
Invalid	
Help	

Definition at line [10](#) of file [shared.hpp](#).

```
00010 {Camera, File, Invalid, Help};
```

4.1.2 Function Documentation

4.1.2.1 parseChoice()

`Choice` Shared::parseChoice (
 std::string s)

Definition at line 8 of file [shared.cpp](#).

```
00008                                     {
00009         //NOTE: Manual Trim
00010         while (!s.empty() && std::isspace((unsigned char)s.front())) s.erase(s.begin());
00011         while (!s.empty() && std::isspace((unsigned char)s.back())) s.pop_back();
00012
00013         if (s.size() != 1) return Choice::Invalid;
00014
00015         char c = (char)std::toupper((unsigned char)s[0]);
00016         if (c == 'C' || c == 'c') return Choice::Camera;
00017         if (c == 'F' || c == 'f') return Choice::File;
00018         if (c == 'H' || c == 'h') return Choice::Help;
00019         return Choice::Invalid;
00020     }
```

Chapter 5

Class Documentation

5.1 Detection Struct Reference

```
#include <marco_annunziata.hpp>
```

Public Attributes

- `cv::Rect` `boundingBox` = `cv::Rect(0,0,0,0)`
- `float` `classConfidence` = 0
- `int` `classId` = -1

5.1.1 Detailed Description

Definition at line 16 of file [marco_annunziata.hpp](#).

5.1.2 Member Data Documentation

5.1.2.1 boundingBox

```
cv::Rect Detection::boundingBox = cv::Rect(0,0,0,0)
```

Definition at line 17 of file [marco_annunziata.hpp](#).

5.1.2.2 classConfidence

```
float Detection::classConfidence = 0
```

Definition at line 18 of file [marco_annunziata.hpp](#).

5.1.2.3 classId

```
int Detection::classId = -1
```

Definition at line 19 of file [marco_annunziata.hpp](#).

The documentation for this struct was generated from the following file:

- include/[marco_annunziata.hpp](#)

5.2 Detections Struct Reference

```
#include <marco_annunziata.hpp>
```

Public Attributes

- `std::vector< cv::Rect >` [boundingBoxes](#)
- `std::vector< float >` [classConfidences](#)
- `std::vector< int >` [classIds](#)

5.2.1 Detailed Description

Definition at line 22 of file [marco_annunziata.hpp](#).

5.2.2 Member Data Documentation

5.2.2.1 boundingBoxes

```
std::vector<cv::Rect> Detections::boundingBoxes
```

Definition at line 23 of file [marco_annunziata.hpp](#).

5.2.2.2 classConfidences

```
std::vector<float> Detections::classConfidences
```

Definition at line 24 of file [marco_annunziata.hpp](#).

5.2.2.3 classIds

```
std::vector<int> Detections::classIds
```

Definition at line 25 of file [marco_annunziata.hpp](#).

The documentation for this struct was generated from the following file:

- include/[marco_annunziata.hpp](#)

5.3 MetricsCounter Struct Reference

```
#include <hermann_serain.hpp>
```

Public Attributes

- int [tp](#) = 0
- int [fp](#) = 0
- int [fn](#) = 0

5.3.1 Detailed Description

Definition at line [27](#) of file [hermann_serain.hpp](#).

5.3.2 Member Data Documentation

5.3.2.1 fn

```
int MetricsCounter::fn = 0
```

Definition at line [30](#) of file [hermann_serain.hpp](#).

5.3.2.2 fp

```
int MetricsCounter::fp = 0
```

Definition at line [29](#) of file [hermann_serain.hpp](#).

5.3.2.3 tp

```
int MetricsCounter::tp = 0
```

Definition at line [28](#) of file [hermann_serain.hpp](#).

The documentation for this struct was generated from the following file:

- include/[hermann_serain.hpp](#)

5.4 TerminalRawMode Struct Reference

Public Member Functions

- [TerminalRawMode](#) ()
- [~TerminalRawMode](#) ()

Public Attributes

- `termios` [oldt](#)

5.4.1 Detailed Description

Definition at line 31 of file [hermann_serain.cpp](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 TerminalRawMode()

```
TerminalRawMode::TerminalRawMode () [inline]
```

Definition at line 35 of file [hermann_serain.cpp](#).

```
00036     {  
00037         tcgetattr(STDIN_FILENO, &oldt);  
00038         termios newt = oldt;  
00039  
00040         newt.c_lflag &= ~(ICANON | ECHO); // no buffer, no echo  
00041         tcsetattr(STDIN_FILENO, TCSANOW, &newt);  
00042     }
```

5.4.2.2 ~TerminalRawMode()

```
TerminalRawMode::~~TerminalRawMode () [inline]
```

Definition at line 44 of file [hermann_serain.cpp](#).

```
00045     {  
00046         tcsetattr(STDIN_FILENO, TCSANOW, &oldt);  
00047     }
```

5.4.3 Member Data Documentation

5.4.3.1 oldt

```
termios TerminalRawMode::oldt
```

Definition at line 33 of file [hermann_serain.cpp](#).

The documentation for this struct was generated from the following file:

- [src/hermann_serain.cpp](#)

5.5 UserData Struct Reference

```
#include <hermann_serain.hpp>
```

Public Member Functions

- [UserData](#) ([Choice](#) _choice, string _data_path)

Public Attributes

- [Choice](#) choice
- string [data_path](#)

5.5.1 Detailed Description

Definition at line 19 of file [hermann_serain.hpp](#).

5.5.2 Constructor & Destructor Documentation

5.5.2.1 UserData()

```
UserData::UserData (  
    Choice _choice,  
    string _data_path) [inline]
```

Definition at line 23 of file [hermann_serain.hpp](#).

```
00024      : choice(_choice), data\_path(_data_path) {}
```

5.5.3 Member Data Documentation

5.5.3.1 choice

```
Choice UserData::choice
```

Definition at line 20 of file [hermann_serain.hpp](#).

5.5.3.2 data_path

```
string UserData::data_path
```

Definition at line 21 of file [hermann_serain.hpp](#).

The documentation for this struct was generated from the following file:

- include/[hermann_serain.hpp](#)

5.6 YOLO_model Class Reference

```
#include <marco_annunziata.hpp>
```

Public Member Functions

- [YOLO_model](#) ()
Constructor that handles the creation of the ONNX Runtime environment.
- bool [isAvailableGPU](#) ()
Checks if GPU providers are available and updates the class with this information.
- std::vector< [Detection](#) > [detectionPipeline](#) (cv::Mat &img, bool enable_letterbox_padding=true)
High-level function for fast and accurate detection of bounding boxes on the given image.
- void [clearDetections](#) ()
- [Detections](#) [detect](#) (const cv::Mat &img, bool enable_letterbox_padding=true)
Detects bounding boxes from an image.
- void [mergeDetections](#) ([Detections](#) &dest, const [Detections](#) &source)
Merges two [Detections](#) structs together in the first struct passed.
- std::vector< std::string > [getDataClasses](#) (std::string labelsFilename="../data/model/labels.txt")
Function to call to read the class names from the labels file and save them inside [classNames](#) attribute of the class.
- std::vector< [Detection](#) > [filterDetectionsNMS](#) ([Detections](#) goodDetections)
Applies Non-Maximum Suppression to detections with low confidence.
- cv::Mat [drawBoundingBoxes](#) (cv::Mat &img, std::vector< [Detection](#) > &detections, cv::Scalar color)
Draws bounding boxes for the detected objects.
- cv::Mat [drawBoundingBoxes](#) (cv::Mat &img, cv::Scalar color)
Draws bounding boxes for the detected objects, using the detections contained in [detections](#).
- void [setModelName](#) (std::string modelName)
modelName setter
- std::string [getModelName](#) ()
modelName getter
- std::vector< [Detection](#) > [getDetections](#) ()
detections getter
- std::vector< std::string > [getDetectionsClassNames](#) ([Detections](#) detections)
Returns the class names of the corresponding id's given by the [Detections](#) struct given in input.
- std::string [getClassName](#) (int classId)
Returns the class name of the corresponding id given in input.

Static Private Member Functions

- static void [logger](#) (void *param, OrtLoggingLevel severity, const char *category, const char *logid, const char *code_location, const char *message)
Logger function used by the constructor that handles the creation of the ONNX Runtime environment.

Private Attributes

- Ort::Env [env](#)
- Ort::Session [session](#)
- Ort::SessionOptions [sessionOptions](#)
- Ort::AllocatorWithDefaultOptions [allocator](#)
- bool [usingGPU](#) = false
- const int [YOLO_TARGET_INPUT_SIZE](#) = 640
- std::vector< [Detection](#) > [detections](#)
- std::string [modelName](#) = "Yolo"
- std::vector< std::string > [classNames](#)
- const float [CLASS_CONFIDENCE_THRESHOLD](#) = 0.5
- const float [NMS_THRESHOLD](#) = 0.5

5.6.1 Detailed Description

Definition at line 28 of file [marco_annunziata.hpp](#).

5.6.2 Constructor & Destructor Documentation

5.6.2.1 YOLO_model()

```
YOLO_model::YOLO_model ()
```

Constructor that handles the creation of the ONNX Runtime environment.

The constructor:

- make use of a logger function for logging its activities
- sets the use of the GPU if present
- sets the number of threads to use as the maximum number available
- sets the path of the model

See also

[YOLO_model::logger\(void* param, OrtLoggingLevel severity, const char* category, const char* logid, const char* code_](#)

Author

Marco Annunziata

Definition at line 27 of file [marco_annunziata.cpp](#).

```
00027         : env(ORT_LOGGING_LEVEL_VERBOSE, "YOLOModel", YOLO_model::logger, this),
    session(nullptr), sessionOptions() {
00028     if(usingGPU){
00029         try{
00030             OrtCUDAProviderOptions cudaOptions;
00031             cudaOptions.device_id = 0;
00032             cudaOptions.gpu_mem_limit = SIZE_MAX;
00033             // arena_extend_strategy: how much space to allocate at a time (arena = allocated block)
00034             // = 0 -> kNextPowerOfTwo, if the arena is filled up double its size (fast but
    RAM-intensive)
00035             // = 1 -> kSameAsRequested, if the arena is filled up increase size exactly to what is
    needed for the operation
00036             cudaOptions.arena_extend_strategy = 0;
00037             sessionOptions.AppendExecutionProvider_CUDA(cudaOptions);
00038             std::cout << "Using ONNX Runtime with GPU resources" << std::endl;
00039         }
00040         catch(const std::exception &e){
00041             std::cout << "CUDA not available! Using ONNX Runtime with CPU resources" << std::endl;
00042         }
00043     }
00044     unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00045     sessionOptions.SetIntraOpNumThreads(maxHardwareThreads);
00046     sessionOptions.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_EXTENDED);
00047     sessionOptions.SetLogId("YOLOModel");
00048
00049     const char* modelPath = "../data/model/YOLO11s_big_best_dynamic.onnx"; // YOLO11s_big_best_dynamic
00050     session = Ort::Session(env, modelPath, sessionOptions);
00051 }
```

5.6.3 Member Function Documentation

5.6.3.1 clearDetections()

```
void YOLO_model::clearDetections ()
```

clears the contents of the class attribute [detections](#)

Definition at line 207 of file [marco_annunziata.cpp](#).

```
00207         {  
00208     this->detections.clear();  
00209 }
```

5.6.3.2 detect()

```
Detections YOLO_model::detect (  
    const cv::Mat & img,  
    bool enable_letterbox_padding = true)
```

Detects bounding boxes from an image.

There is a pre-processing step to prepare the image for the YOLO model (including letterbox padding, if enabled).

Attention

- 1) The function by itself doesn't clear previous detections saved inside the class.
- 2) The function returns ALL detections, even the ones with low confidence.

Parameters

<i>img</i>	A cv::Mat object containing the image
<i>enable_letterbox_padding</i>	flag to enable letterbox padding on the image before running the detection

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

[clearDetections\(\)](#)

[filterDetectionsNMS\(Detections goodDetections\)](#)

Author

Marco Annunziata

Definition at line 243 of file marco_annunziata.cpp.

```

00243                                     {
00244     /* Note: there is small to no documentation of the C++ implementation of OnnxRuntime even from the
00245     developers themselves, so this function has been thoroughly optimized and documented
00246     such that every step is perfectly clear to everyone reading this code. */
00247     int inputHeight = img.rows;
00248     int inputWidth  = img.cols;
00249
00250     /* It's possible to give an image as input without resizing in OnnxRuntime because it supports
00251     dynamic input, but the processing of the model is way slower.
00252     The input needs to be resized anyway for the YOLO model to have good accuracy, so
00253     pre-processing of the image is still needed and the dynamic input feature is not used for YOLO */
00254     /* YOLO: PRE-PROCESSING required for the image
00255     the YOLO model requires the image to be:
00256     1. Of dimensions multiples of 32, because we know YOLO has a CNN in its architecture:
00257     -There are 5 convolutional layers with stride=2 where the convolutions downsample
00258     the input by a factor of 2^5=32, so the feature map has size inputSize/32
00259     -The stride of convolution replaces max pooling of typical CNNs
00260     -If the inputSize is not divisible by 32, the feature map become of size
00261     non-integer which leads to feature misalignment
00262     2. Of overall size square (multiple of 32), typically 640x640 or 416x416
00263     3. not deformed (deformed = stretched image), since YOLO doesn't recognize deformed
00264     objects and, even if you train it for that, the predictions are still made in fixed grid cells
00265     This means that:
00266     - If an image is smaller or bigger than 640x640, it has to be resized to the same scale of
00267     640x640 (requirement 2)
00268     - If the image is not square, you need to resize it while keeping its aspect ratio w:h, so
00269     that the result is not deformed (requirement 3)
00270     => the scaling factor has to be saved so that you can correctly resize the bounding box
00271     to the original size of the image
00272     - If the image is not square, after resizing you also need to add padding to have a
00273     resulting image of exactly 640x640 size (requirement 2)
00274     So the pre-processing of the images is composed of these steps:
00275     Case A: If the image is square, just resize it if needed and you're done
00276     Case B: If the image is not square (most of the time):
00277     1. Resize the image to a multiple of 32 while keeping its aspect ratio, which means:
00278     resize the longest side to 640 and adjust the shorter side following proportions w:h
00279     2. add padding to the shorter side in order to reach the square size.
00280     Note: to guarantee the best performance, the padding has to be done in the same we YOLO was
00281     trained for.
00282     YOLO was trained using images with letterbox padding, which means that the image is centered
00283     and the padding is around the image.
00284     This is useful because in the uncentered padding the model would have a small bias of
00285     detecting objects which are closer to the corner.
00286     A symmetric padding (letterbox) is more robust to this positional bias.
00287     */
00288     Mat resizedImg;
00289     float resizedWidth  = -1;
00290     float resizedHeight = -1;
00291     int xPaddedImgCorner = 0; // It represents the upper left corner of the centered image after
00292     letterbox padding, will be useful later to center the image
00293     int yPaddedImgCorner = 0;
00294
00295     float imgAspectRatio = static_cast<float>(inputWidth) / inputHeight;
00296     if(inputWidth == inputHeight){
00297     /* Case A: image is square, just resize */
00298     resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00299     resizedHeight = YOLO_TARGET_INPUT_SIZE;
00300     resize(img, resizedImg, Size(static_cast<int>(resizedWidth),
00301     static_cast<int>(resizedHeight)));
00302     }
00303     else{
00304     /* Case B: image is not square. Resize following aspect ratio, then add letterbox padding*/
00305     if(imgAspectRatio > 1.0f){
00306     resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00307     resizedHeight = YOLO_TARGET_INPUT_SIZE / imgAspectRatio; // imgAspectRatio > 1 -> have to
00308     divide so that the height follows the width increase/decrease
00309     resizedHeight = (static_cast<int>(resizedHeight) / 32) * 32; // Ensure dimensions of the
00310     image are multiples of 32, as required by YOLO11 (in this case width is already multiple of 32)
00311
00312     if(enable_letterbox_padding == true) yPaddedImgCorner =
00313     static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedHeight/2);
00314     }
00315     else{
00316     resizedHeight = YOLO_TARGET_INPUT_SIZE;
00317     resizedWidth  = YOLO_TARGET_INPUT_SIZE * imgAspectRatio; // imgAspectRatio < 1 -> have to
00318     multiply to make the height follow the width value
00319     resizedWidth  = (static_cast<int>(resizedWidth) / 32) * 32;

```

```

00303
00304         if(enable_letterbox_padding == true) xPaddedImgCorner =
static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedWidth/2);
00305     }
00306
00307     Mat tempImg;
00308     resize(img, tempImg, Size(static_cast<int>(resizedWidth), static_cast<int>(resizedHeight)));
00309
00310     //letterbox padding
00311     if(enable_letterbox_padding == true){
00312         resizedImg = Mat::zeros(YOLO_TARGET_INPUT_SIZE, YOLO_TARGET_INPUT_SIZE, CV_8UC3);
00313         tempImg.copyTo(resizedImg(Rect(xPaddedImgCorner, yPaddedImgCorner, resizedWidth,
resizedHeight)));
00314     }
00315     else resizedImg = tempImg.clone();
00316 }
00317
00318 float resizeScalingFactor[2] = {
00319     static_cast<float>(inputWidth) / resizedWidth,
00320     static_cast<float>(inputHeight) / resizedHeight
00321 };
00322
00323 /* TENSORS
00324 OnnxRuntime needs a tensor as input.
00325 The ONNX tensor is not the mathematical tensor (although it is inspired by it).
00326 It's a multi-dimensional array of heterogeneous data types (float, int ...) which is optimized for
computations.
00327 Tensors are useful because they are a convenient data structure for parallelizing GPU computations
during DNN inference.*/
00328
00329 /* INPUT TENSOR
00330 * In OnnxRuntime the tensor is defined by its data (an array) and its shape (dimensions of the
multi-dimensional array)
00331 * Note: ONNX has the convention to have the nput tensor shape following the NCHW order, where N =
batch size, C = n°channels, H = height, W = width of the image.
00332 * For convention height comes before width because data in memory is accessed along the row:
data[y][x]
00333 * You can check the shape of input tensor of your model by inspecting the .onnx file with
https://netron.app/
00334 */
00335
00336 /* Preparing the parameters for the dynamic INPUT TENSOR */
00337 vector<int64_t> inputShape = {1, 3, static_cast<int>(resizedHeight),
static_cast<int>(resizedWidth)}; // 4D tensor
00338 size_t inputTensorSize = 1 * 3 * resizedHeight * resizedWidth;
00339 vector<float> inputData(inputTensorSize); // the input image is flattened into a mono-dimensional
array
00340
00341 // Converting image data from cv::Mat to tensor (normalization to [0,1] and converting (cv::Mat)
BGR to RGB as the YOLO model requires)
00342 double normalizationFactor = 1.0 / 255.0;
00343 for (int row = 0; row < resizedHeight; row++) {
00344     for (int col = 0; col < resizedWidth; col++) {
00345         Vec3b pixel = resizedImg.at<Vec3b>(row, col);
00346         // cv::Mat -> Tensor
00347         inputData[2 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[0] *
normalizationFactor; // B -> R
00348         inputData[1 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[1] *
normalizationFactor; // G -> G
00349         inputData[0 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[2] *
normalizationFactor; // R -> B
00350     }
00351 }
00352
00353 auto memoryInfo = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault); // Allocating
the input tensor on CPU
00354
00355 // Creating the input tensor (which is a view of the data, NOT a deep copy!)
00356 Ort::Value inputOnnxTensor = Ort::Value::CreateTensor<float>(memoryInfo, inputData.data(),
inputTensorSize, inputShape.data(), inputShape.size());
00357
00358 // retrieving now as string the name of the first input and output nodes
00359 // the allocations of these strings are automatically freed by the session
00360 auto inputNodeName = session.GetInputNameAllocated(0, allocator);
00361 auto outputNodeName = session.GetOutputNameAllocated(0, allocator);
00362
00363 // The API needs the array of inputs to set and the array of outputs to get (or even just a
portion of output if needed).
00364 // So here you are setting the ORDER of the input tensors and the ORDER of the outputs.
00365 // This step is useful for models which have multiple input and output tensors where the order
matters.
00366 // In our case we just have a 1 to 1 correspondence of input-output so the arrays passed will only
have 1 item
00367 vector<const char*> inputNodeNames = {inputNodeName.get()};
00368 vector<const char*> outputNodeNames = {outputNodeName.get()};
00369
00370 // Running the inference

```

```

00370     auto outputTensors = session.Run(
00371         Ort::RunOptions{nullptr}, // e.g set a verbosity level only for this run
00372         inputNodeNames.data(), &inputOnnxTensor, inputNodeNames.size(), // input to set (=1 input
image)
00373         outputNodeNames.data(), // output to take
00374         outputNodeNames.size() // actually = 1 output tensors because YOLO has only 1
output layer
00375     );
00376
00377     /* OUTPUT TENSOR
00378     // YOLOl1 is pre-trained by default on the COCO dataset, so the number of channels should be:
00379     //                                     84 = 4 + 80 (= class confidence, one for each class)
00380     // BUT, we fine-tuned the training of YOLOl1 with our custom dataset, so the output layer is
restructured to represent 53 classes.
00381     // This means the number of channels should be 57 = 4 (bounding box parameters) + 53 (= class
confidence, one for each class)
00382     // Note: class probability = probability of having detected one class in that region
00383     //       class confidence = confidence of having detected an object of a class in that
region, measured as class probability * objectness probability
00384     //       objectness probability = probability of having an object in that region, useful for
YOLOv5 but not for YOLOl1, since there is already class confidence, so it's been removed.
00385
00386     // Note, YOLOv5 has a different representation of the output layer compared to YOLOv8/YOLOl1:
00387     // YOLOv5 -> channels = 85 = 4 (bb_parameters) + 1 (objectness probability) + 80 (= class
probabilities of corresponding 80 classes of the COCO dataset)
00388     // So, in this case the objectness is separate from class confidence and each class has the class
probability instead of the class confidence.
00389     // To get class confidence of each class you have to multiply each class probability*objectness.
So in YOLOv5 the objectness probability was useful!
00390
00391     // To recap, our custom YOLOl1 output tensor (3D Mat) has shape (1, 57, 8400*):
00392     // 1 - is the batch size. If =0, it can change dynamically depending
on the input of the net (not needed since in YOLO batch size = 1).
00393     // 57 (84 for default model) - is the number of channels, where the first 4 are the bounding
box parameters (x, y, width, height)
00394     //                                     and the other ones are class probabilities for COCO'S 80
classes (There is no objectiveness parameter in YOLOv8/l1)
00395     // 8400 *or lower - is the total number of regions where the predictions are made
("regions" = "grid points" = anchor locations).
00396     //                                     *Since the ONNX model is dynamic, the output tensor can have
a lower number of detections depending on how big is the padding area.
00397     // The first and the third value become 0 when exported into ONNX format with dynamic=True,
indicating that in the dynamic model these parameters can change on runtime.
00398     // Dynamic input/output shapes are not supported by OpenCV, but they are supported by ONNX
Runtime.
00399     // In this case, OnnxRuntime is adopted instead of OpenCV more because of the superior inference
efficiency rather than the support of dynamic models.
00400     // You can also see the shape of the output layer by importing the .onnx file in
https://netron.app/
00401     // See also: https://github.com/orgs/ultralytics/discussions/17254
00402     */
00403
00404     /* Getting dynamic output tensor info (data, shape, count) */
00405     auto& outputTensor = outputTensors[0]; // In the case of YOLOl1, only 1 output layer is used, so
there is only 1 output tensor (outputTensors[0])
00406     auto tensorInfo = outputTensor.GetTensorTypeAndShapeInfo();
00407     auto tensorCount = tensorInfo.GetElementCount(); UNUSED(tensorCount);
00408     auto tensorShape = tensorInfo.GetShape();
00409     float* outputData = outputTensor.GetTensorMutableData<float>(); // the output values are flattened
into a 1D array
00410     // YOLOl1: outputData[0..4...] = (x, y, width, height, confidence, classConfidence1,
classConfidence2, ...)
00411
00412     int batchSize = static_cast<int>(tensorShape[0]); UNUSED(batchSize);
00413     int numChannels = static_cast<int>(tensorShape[1]);
00414     int numDetections = static_cast<int>(tensorShape[2]); // The number of detections will vary based
on input size and padding area
00415
00416     // NOTE: The output tensor is a matrix like this, if the batch size =1: (neglecting the first four
parameters used for the bounding box)
00417     //         class1_detection1      class1_detection2      ...      class1_detectionN
00418     //         ....
00419     //         class57_detection1      class57_detectionN
00420
00421     Detections goodDetections;
00422     vector<int> classIds;
00423     vector<float> detectedClassConfidences;
00424     vector<Rect> boundingBoxes;
00425
00426     /* Declaring things used in the post-process phase */
00427     Mat classScores;
00428     Point classId;
00429     double maxClassScore;
00430
00431     // I want to iterate through the output as a flat 1D array, so that i can use a pointer instead of
copying values
00432     // When doing this, I need the output to have detections as rows and classes as columns so that I

```

```

    can evaluate the bounding box of each detection.
00433 //      class1_detection1      class1_detection2      ...      class1_detectionN
TRANPOSE      detection1_class1      detection1_class2      ...      detection1_class57
00434 //      ....
--->      ....
00435 //      class57_detection1      class57_detectionN
detectionN_class1      detectionN_class57
00436 // This is why a Mat is constructed from the pointer outputData and its transposition t() is taken
00437 Mat outputDataMat= Mat(numChannels, numDetections, CV_32F, outputData).t(); // -> now the data is
in the order we needed! [Det1Class1 Det1Class2 ... Det1Class57 Det2Class1 ...]
00438 // -> This Mat constructor with the fourth parameter is defined as:
00439 // Mat(int rows, int cols, int type, void* data, size_t step=AUTO_STEP);
00440 // The fourth parameter is just a pointer to the data, it doesn't need to copy the values!
00441 // So this constructor just wraps pointer outputData in a matrix header, resulting a lot more
efficient.

00442
00443 for(int i = 0; i < numDetections; i++){
00444     float* currentDetection = outputDataMat.ptr<float>(i); // 1 detection corresponds to a 1D
array of the 57 elements of the detection
00445     float* firstClassScore = currentDetection + 4;
00446     classScores = Mat(1, numChannels-4, CV_32FC1, firstClassScore); // using efficient constructor
00447     // Also, minMaxLoc accepts as input a Mat and as output a Point, so this is why classScores
and classId are defined as Mat and Point.

00448
00449     minMaxLoc(classScores, 0, &maxClassScore, 0, &classId);
00450
00451     if(maxClassScore > CLASS_CONFIDENCE_THRESHOLD){
00452         detectedClassConfidences.push_back(maxClassScore);
00453         classIds.push_back(classId.x);
00454
00455         float xCenter = currentDetection[0];
00456         float yCenter = currentDetection[1];
00457         float boxWidth = currentDetection[2];
00458         float boxHeight = currentDetection[3];
00459
00460         /* removing coordinates translation caused by the padding */
00461         if(enable_letterbox_padding == true){
00462             xCenter -= xPaddedImgCorner;
00463             yCenter -= yPaddedImgCorner;
00464         }
00465
00466         /* computing the bounding box of the detected object and scaling it to the original image
size */
00467         int xPosScaled = int((xCenter - 0.5f*boxWidth) * resizeScalingFactor[0]);
00468         int yPosScaled = int((yCenter - 0.5f*boxHeight) * resizeScalingFactor[1]);
00469         int bWScaled = int(boxWidth * resizeScalingFactor[0]);
00470         int bHScaled = int(boxHeight * resizeScalingFactor[1]);
00471
00472         Rect box = Rect(xPosScaled, yPosScaled, bWScaled, bHScaled);
00473
00474         /* Clamping bounding box to image boundaries */
00475         box.x = max(0, box.x);
00476         box.y = max(0, box.y);
00477         box.width = min(box.width, img.cols - box.x);
00478         box.height = min(box.height, img.rows - box.y);
00479
00480         boundingBoxes.push_back(box);
00481     }
00482 }
00483
00484 goodDetections.boundingBoxes = boundingBoxes;
00485 goodDetections.classConfidences = detectedClassConfidences;
00486 goodDetections.classIds = classIds;
00487
00488 return goodDetections;
00489 /* Non-Maxima Suppression and construction of the final array of detections */
00490 }

```

5.6.3.3 detectionPipeline()

```

std::vector< Detection > YOLO_model::detectionPipeline (
    cv::Mat & img,
    bool enable_letterbox_padding = true)

```

High-level function for fast and accurate detection of bounding boxes on the given image.

The pipeline consists of:

- clearing the previous model detections

- checking if the GPU is available
- using a multi-threaded tiled approach for detecting bounding boxes in big images (which are images with one dimension exceeds 1.5*640 pixels)
- directly starting the detection for images small enough (with all dimensions within 1.5*640 pixels)
- filtering the detections using Non-Maximum Suppression (NMS)

Parameters

<i>img</i>	A cv::Mat object containing the image
<i>enable_letterbox_padding</i>	flag to enable letterbox padding on the image before running the detection

Returns

a std::vector of [Detection](#) structs with all good detections found in the image.

See also

[detect](#)(const Mat &img, bool enable_letterbox_padding)
[mergeDetections](#)(Detections& dest, const Detections& source)
[filterDetectionsNMS](#)(Detections goodDetections)

Author

Marco Annunziata

Definition at line 134 of file [marco_annunziata.cpp](#).

```

00134                                     {
00135     int slice_size = 640;
00136     float overlap_ratio = 0.2;
00137     int step = slice_size * (1 - overlap_ratio);
00138
00139     // removes previous detections, in case of multiple subsequent detections
00140     clearDetections();
00141
00142     this->isAvailableGPU();
00143
00144     if(img.cols > 1.5*slice_size || img.rows > 1.5*slice_size){
00145         Detections mergedDetections;
00146         /* Tiled approach if image is too big (makes cards easier to detect for the model) */
00147         if(this->usingGPU){
00148             Detections tileDetections;
00149             for(int y = 0; y <= img.rows - slice_size; y += step){
00150                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00151                     cv::Rect roi(x, y, slice_size, slice_size);
00152                     cv::Mat tile = img(roi);
00153                     tileDetections = this->detect(tile, enable_letterbox_padding);
00154                     /* proceeding to map the box coordinates found in the tile to the whole image,
00155                      * then adding the detections of the specific tile to the detections of the overall
00156                      image
00157                      */
00158                     for(auto &box : tileDetections.boundingBoxes){
00159                         box.x += x;
00160                         box.y += y;
00161                     }
00162                     mergeDetections(mergedDetections, tileDetections);
00163                 }
00164             }
00165             /* Detection on the whole image improves result for large images having really large cards
00166             */
00167             mergeDetections(mergedDetections, this->detect(img, enable_letterbox_padding));
00168             this->detections = filterDetectionsNMS(mergedDetections);
00169         }
00170         else{
00171             unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00172             auto activeThreads = std::make_shared<std::atomic<int>>(0);
00173             std::vector<std::future<Detections>> futures;
00174
00175             for(int y = 0; y <= img.rows - slice_size; y += step){
00176                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00177                     while((*activeThreads) >= (int) maxHardwareThreads){
00178                         std::this_thread::yield();
00179                     }
00180                     futures.push_back(std::async(std::launch::async, [this, img, x, y, slice_size, enable_letterbox_padding]{
00181                         return detect(cv::Mat(img, cv::Rect(x, y, slice_size, slice_size)), enable_letterbox_padding);
00182                     }));
00183                     (*activeThreads)++;
00184                 }
00185             }
00186             for(auto &future : futures){
00187                 futures.push_back(future.get());
00188             }
00189             this->detections = filterDetectionsNMS(futures);
00190         }
00191     }
00192 }

```

```

00177         }
00178
00179         (*activeThreads)++;
00180         cv::Rect roi(x, y, slice_size, slice_size);
00181         cv::Mat tile = img(roi).clone();
00182
00183         futures.push_back(std::async(std::launch::async, [this, tile,
enable_letterbox_padding, x, y, activeThreads]() {
00184             Detections tileDetections = this->detect(tile, enable_letterbox_padding);
00185             /* proceeding to map the box coordinates found in the tile to the whole image,
00186              * then adding the detections of the specific tile to the detections of the
overall image */
00187             for(auto &box : tileDetections.boundingBoxes){
00188                 box.x += x;
00189                 box.y += y;
00190             }
00191             (*activeThreads)--;
00192             return tileDetections;
00193         }));
00194     }
00195 }
00196 /* Detect on the whole image, collect detections and merge them, then filter with NMS
suppression */
00197 futures.push_back(std::async(std::launch::async, [this, img, enable_letterbox_padding]() {
return this->detect(img, enable_letterbox_padding); }));
00198 for(auto& future : futures) mergeDetections(mergedDetections, future.get());
00199 this->detections = filterDetectionsNMS(mergedDetections);
00200 }
00201 }
00202 else this->detections = filterDetectionsNMS(this->detect(img, enable_letterbox_padding));
00203 return this->detections;
00204 }

```

5.6.3.4 drawBoundingBoxes() [1/2]

```

cv::Mat YOLO_model::drawBoundingBoxes (
    cv::Mat & img,
    cv::Scalar color)

```

Draws bounding boxes for the detected objects, using the detections contained in [detections](#).

The bounding box is scaled depending on the input image size, using empirical values.

Parameters

<i>img</i>	a integer representing the id of the detected bounding box
<i>color</i>	a integer representing the id of the detected bounding box

Returns

A cv::Mat object containing the image with bounding box drawn, if present.

See also

`drawBoundingBoxes(Mat &img, std::vector<Detection> &detections, Scalar color)`

Author

Marco Annunziata

Definition at line 589 of file [marco_annunziata.cpp](#).

```

00589     {
00590         return drawBoundingBoxes(img, this->detections, color);
00591     }

```


5.6.3.5 drawBoundingBoxes() [2/2]

```
cv::Mat YOLO_model::drawBoundingBoxes (
    cv::Mat & img,
    std::vector< Detection > & detections,
    cv::Scalar color)
```

Draws bounding boxes for the detected objects.

The bounding box is scaled depending on the input image size, using empirical values.

Parameters

<i>img</i>	a integer representing the id of the detected bounding box
<i>detections</i>	a vector of Detection structs containing the detections
<i>color</i>	a integer representing the id of the detected bounding box

Returns

A cv::Mat object containing the image with bounding box drawn, if present.

See also

[drawBoundingBoxes\(Mat &img, Scalar color\)](#)

Author

Marco Annunziata

Definition at line 565 of file [marco_annunziata.cpp](#).

```
00565
00566     Mat resultImg = img.clone();
00567     for (auto detection : detections)
00568     {
00569         int thickness = max(1, int(max(img.rows, img.cols) / 640));
00570         rectangle(resultImg, detection.boundingBox, color, 2 * thickness);
00571         string label = this->getClassName(detection.classId) + " " +
00572             to_string(static_cast<int>(detection.classConfidence * 100)) + "%";
00572         putText(resultImg, label, Point(detection.boundingBox.x, detection.boundingBox.y - 5 *
00573             thickness), FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 * thickness);
00573     }
00574     // std::string windowTitle = getModelName() + " - " + std::to_string(detections.size()) + "
00575     detections";
00575     // namedWindow(windowTitle, WINDOW_NORMAL);
00576     // imshow(windowTitle, resultImg);
00577     // waitKey(0);
00578     return resultImg;
00579 }
```

5.6.3.6 filterDetectionsNMS()

```
std::vector< Detection > YOLO_model::filterDetectionsNMS (
    Detections goodDetections)
```

Applies Non-Maximum Suppression to detections with low confidence.

Parameters

<i>goodDetections</i>	a Detections struct containing the detections
-----------------------	---

Returns

A std::vector of [Detection](#) structs containing only the detections with confidence higher than [CLASS_CONFIDENCE_THRESHOLD](#)

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

Author

Marco Annunziata

Definition at line 498 of file [marco_annunziata.cpp](#).

```

00498                                     {
00499     vector<int> resultNMS;
00500     cv::dnn::NMSBoxes(goodDetections.boundingBoxes, goodDetections.classConfidences,
00501                       CLASS_CONFIDENCE_THRESHOLD, NMS_THRESHOLD, resultNMS);
00502     std::vector<Detection> finalDetections;
00503     for(auto finalBoxIndex : resultNMS){
00504         Detection finalDetection;
00505         finalDetection.classId = goodDetections.classIds[finalBoxIndex];
00506         finalDetection.classConfidence = goodDetections.classConfidences[finalBoxIndex];
00507         finalDetection.boundingBox = goodDetections.boundingBoxes[finalBoxIndex];
00508         finalDetections.push_back(finalDetection);
00509     }
00510     return finalDetections;
00511 }
```

5.6.3.7 getClassNames()

```

std::string YOLO_model::getClassNames (
    int classId)
```

Returns the class name of the corresponding id given in input.

In order to find names, [classNames](#) is used, meaning it must not be empty.

Attention

Function to call only when class names have already been read from the file, meaning [getDataClasses\(string labelsFilename\)](#) has to be called first

Parameters

<i>classId</i>	a integer representing the id of the detected bounding box
----------------	--

Returns

A string containing the corresponding name

Author

Marco Annunziata

Definition at line 552 of file [marco_annunziata.cpp](#).

```

00552                                     {
00553     return this->classNames[classId];
00554 }
```

5.6.3.8 getDataClasses()

```
vector< string > YOLO_model::getDataClasses (
    std::string labelsFilename = "../data/model/labels.txt")
```

Function to call to read the class names from the labels file and save them inside [classNames](#) attribute of the class.

Parameters

<i>labelsFilename</i>	a string with the path of the labels file
-----------------------	---

Returns

A `std::vector` of strings containing the labels saved in [classNames](#)

See also

[getDetectionsClassNames\(Detections detections\)](#)

Author

Marco Annunziata

Definition at line 519 of file [marco_annunziata.cpp](#).

```
00519                                     {
00520     vector<string> dataClasses;
00521     ifstream ifs(labelsFilename);
00522     string line;
00523     while (getline(ifs, line))
00524         dataClasses.push_back(line);
00525     this->classNames = dataClasses;
00526     return dataClasses;
00527 }
```

5.6.3.9 getDetections()

```
std::vector< Detection > YOLO_model::getDetections ()
```

[detections](#) getter

Author

Marco Annunziata

Definition at line 610 of file [marco_annunziata.cpp](#).

```
00610                                     {
00611     return this->detections;
00612 }
```

5.6.3.10 getDetectionsClassNames()

```
vector< string > YOLO_model::getDetectionsClassNames (
    Detections detections)
```

Returns the class names of the corresponding id's given by the [Detections](#) struct given in input.

In order to find names, [classNames](#) is used, meaning it must not be empty.

Attention

Function to call only when class names have already been read from the file, meaning `getDataClasses(string labelsFilename)` has to be called first

Parameters

<i>detections</i>	a Detections struct containing all detections
-------------------	---

Returns

A std::vector of strings containing the corresponding names

See also

`getDataClasses(string labelsFilename)`

Author

Marco Annunziata

Definition at line 537 of file [marco_annunziata.cpp](#).

```
00537                                     {
00538     std::vector<string> detectionsClassNames;
00539     for(int id: detections.classIds){
00540         detectionsClassNames.push_back(this->classNames[id]);
00541     }
00542     return detectionsClassNames;
00543 }
```

5.6.3.11 getModelName()

```
string YOLO_model::getModelName ()
```

[modelName](#) getter

Author

Marco Annunziata

Definition at line 603 of file [marco_annunziata.cpp](#).

```
00603                                     {
00604     return this->modelName;
00605 }
```

5.6.3.12 isAvailableGPU()

```
bool YOLO_model::isAvailableGPU ()
```

Checks if GPU providers are available and updates the class with this information.

Returns

a boolean which is true if a GPU provider is available.

See also

`detectionPipeline(Mat &img, bool enable_letterbox_padding)`

Definition at line 104 of file [marco_annunziata.cpp](#).

```
00104                                     {
00105     // Ottieni la lista dei provider compilati nel tuo ONNX Runtime
00106     std::vector<std::string> providers = Ort::GetAvailableProviders();
00107
00108     for (const auto& p : providers) {
00109         if (p == "CUDAXecutionProvider" || p == "DmlExecutionProvider") {
00110             this->usingGPU = true;
00111             return true;
00112         }
00113     }
00114     this->usingGPU = false;
00115     return false;
00116 }
```

5.6.3.13 logger()

```
void YOLO_model::logger (
    void * param,
    OrtLoggingLevel severity,
    const char * category,
    const char * logid,
    const char * code_location,
    const char * message) [static], [private]
```

Logger function used by the constructor that handles the creation of the ONNX Runtime environment.

Parameters

<i>param</i>	
<i>severity</i>	
<i>category</i>	
<i>logid</i>	
<i>code_location</i>	
<i>message</i>	

See also

[YOLO_model::YOLO_model\(\)](#)

Author

Marco Annunziata

Definition at line 64 of file [marco_annunziata.cpp](#).

```
00065     {
00066         static std::mutex logMutex;
00067         static std::ofstream logFile("./onnxruntime.log", std::ios::out | std::ios::app);
00068
00069         const char* severityStr = "";
00070         switch (severity) {
00071             case ORT_LOGGING_LEVEL_VERBOSE: severityStr = "VERBOSE"; break;
00072             case ORT_LOGGING_LEVEL_INFO: severityStr = "INFO"; break;
00073             case ORT_LOGGING_LEVEL_WARNING: severityStr = "WARNING"; break;
00074             case ORT_LOGGING_LEVEL_ERROR: severityStr = "ERROR"; break;
00075             case ORT_LOGGING_LEVEL_FATAL: severityStr = "FATAL"; break;
00076             default: severityStr = "UNKNOWN"; break;
00077         }
00078
00079         // concatenating the output into a single string buffer to write into the log file
00080         std::ostringstream os;
00081         os << "[" << (logid ? logid : "no-logid") << "]" << " "
00082             << severityStr << " | " << (category ? category : "") << " @ "
00083             << (code_location ? code_location : "") << "\n" << " "
00084             << (message ? message : "") << "\n";
00085
00086         std::lock_guard<std::mutex> lock(logMutex);
00087
00088         if (severity <= ORT_LOGGING_LEVEL_INFO) {
00089             // INFO and VERBOSE go to file
00090             if (logFile) {
00091                 logFile << os.str();
00092                 logFile.flush();
00093             }
00094         } else {
00095             // WARNING, ERROR, FATAL go to stderr
00096             std::cerr << os.str();
00097         }
00098     }
```

5.6.3.14 mergeDetections()

```
void YOLO_model::mergeDetections (
    Detections & dest,
    const Detections & source)
```

Merges two [Detections](#) structs together in the first struct passed.

Useful for merging detections of different tiles of the same image.

Parameters

<i>dest</i>	pointer to the Detections struct used as destination
<i>source</i>	pointer to the Detections struct to merge

See also

[detectionPipeline\(Mat &img, bool enable_letterbox_padding\)](#)

Author

Marco Annunziata

Definition at line 218 of file [marco_annunziata.cpp](#).

```
00218                                     {
00219     if(source.boundingBoxes.empty()) return;
00220
00221     size_t newSize = dest.boundingBoxes.size() + source.boundingBoxes.size();
00222
00223     dest.boundingBoxes.reserve(newSize);
00224     dest.classConfidences.reserve(newSize);
00225     dest.classIds.reserve(newSize);
00226
00227     dest.boundingBoxes.insert(dest.boundingBoxes.end(), source.boundingBoxes.begin(),
source.boundingBoxes.end());
00228     dest.classConfidences.insert(dest.classConfidences.end(), source.classConfidences.begin(),
source.classConfidences.end());
00229     dest.classIds.insert(dest.classIds.end(), source.classIds.begin(), source.classIds.end());
00230
00231 }
```

5.6.3.15 setModelName()

```
void YOLO_model::setModelName (
    std::string modelName)
```

[modelName](#) setter

Author

Marco Annunziata

Definition at line 596 of file [marco_annunziata.cpp](#).

```
00596                                     {
00597     this->modelName = modelName;
00598 }
```

5.6.4 Member Data Documentation

5.6.4.1 allocator

```
Ort::AllocatorWithDefaultOptions YOLO_model::allocator [private]
```

Definition at line 34 of file [marco_annunziata.hpp](#).

5.6.4.2 CLASS_CONFIDENCE_THRESHOLD

```
const float YOLO_model::CLASS_CONFIDENCE_THRESHOLD = 0.5 [private]
```

Definition at line 40 of file [marco_annunziata.hpp](#).

5.6.4.3 classNames

```
std::vector<std::string> YOLO_model::classNames [private]
```

Definition at line 39 of file [marco_annunziata.hpp](#).

5.6.4.4 detections

```
std::vector<Detection> YOLO_model::detections [private]
```

Definition at line 37 of file [marco_annunziata.hpp](#).

5.6.4.5 env

```
Ort::Env YOLO_model::env [private]
```

Definition at line 30 of file [marco_annunziata.hpp](#).

5.6.4.6 modelName

```
std::string YOLO_model::modelName = "Yolo" [private]
```

Definition at line 38 of file [marco_annunziata.hpp](#).

5.6.4.7 NMS_THRESHOLD

```
const float YOLO_model::NMS_THRESHOLD = 0.5 [private]
```

Definition at line 41 of file [marco_annunziata.hpp](#).

5.6.4.8 session

```
Ort::Session YOLO_model::session [private]
```

Definition at line 32 of file [marco_annunziata.hpp](#).

5.6.4.9 sessionOptions

```
Ort::SessionOptions YOLO_model::sessionOptions [private]
```

Definition at line 33 of file [marco_annunziata.hpp](#).

5.6.4.10 usingGPU

```
bool YOLO_model::usingGPU = false [private]
```

Definition at line 35 of file [marco_annunziata.hpp](#).

5.6.4.11 YOLO_TARGET_INPUT_SIZE

```
const int YOLO_model::YOLO_TARGET_INPUT_SIZE = 640 [private]
```

Definition at line 36 of file [marco_annunziata.hpp](#).

The documentation for this class was generated from the following files:

- include/[marco_annunziata.hpp](#)
- src/[marco_annunziata.cpp](#)

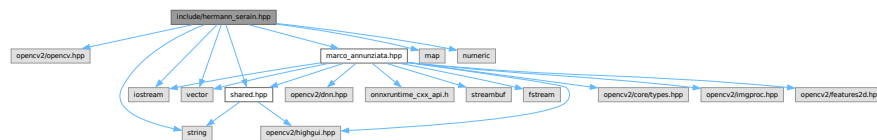
Chapter 6

File Documentation

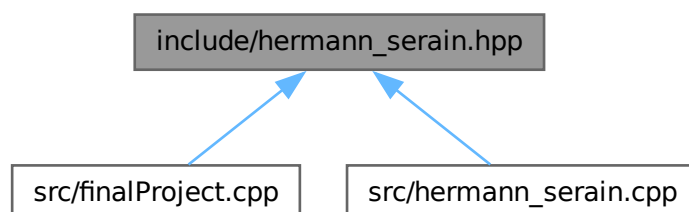
6.1 include/hermann_serain.hpp File Reference

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <numeric>
#include "shared.hpp"
#include "marco_annunziata.hpp"
```

Include dependency graph for hermann_serain.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [UserData](#)
- struct [MetricsCounter](#)

Functions

- [UserData readInput](#) (int argc, char **argv)
- void [computeImageMetrics](#) (string imagePath, std::vector< [Detection](#) > predictedDetections)

6.1.1 Function Documentation

6.1.1.1 computeImageMetrics()

```
void computeImageMetrics (
    string imagePath,
    std::vector< Detection > predictedDetections)
```

Definition at line 438 of file [hermann_serain.cpp](#).

```
00438                                     {
00439     if (!canComputeImageMetrics(imagePath)) {
00440         return;
00441     }
00442
00443     fs::path p(imagePath);
00444     string filename = p.stem().string();
00445     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00446
00447     cv::Mat img = cv::imread(imagePath);
00448     if (img.empty()) {
00449         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00450         return;
00451     }
00452
00453     // Extraction of ground truth using the helper method
00454     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00455
00456     if (!actualDetections.empty()) {
00457         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00458         printFinalMetrics(results);
00459     }
00460 }
```

6.1.1.2 readInput()

```
UserData readInput (
    int argc,
    char ** argv)
```

Definition at line 63 of file [hermann_serain.cpp](#).

```
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG\_VIDEO) {
00073                 result.data_path = DEFAULT\_VIDEO\_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG\_IMAGE) {
00077                 result.data_path = DEFAULT\_IMAGE\_PATH;
00078             }
00079         }
00080     }
00081 }
```

```

00078         result.choice = Choice::File;
00079     }
00080 }
00081 }
00082
00083 if(result.choice != Choice::Invalid)
00084     return result;
00085
00086 //NOTE: I'm gonna read the user input from now on
00087 string choiceStr;
00088
00089 // NOTE: Let's keep asking if the input is invalid
00090 while (result.choice == Choice::Invalid)
00091 {
00092     cout << "-----\n";
00093     cout << " C = Camera | F = File | H = Help\n";
00094     cout << "-----\n";
00095     cout << "Write your choice: ";
00096
00097     cin >> choiceStr;
00098     result.choice = parseChoice(choiceStr);
00099
00100     switch (result.choice)
00101     {
00102         case Choice::Camera:
00103         {
00104             handleCamera();
00105         }
00106         break;
00107
00108         case Choice::File:
00109         {
00110             result.data_path = handleFile();
00111         }
00112         break;
00113
00114         case Choice::Help:
00115         {
00116             printHelp();
00117             result.choice = Choice::Invalid; //NOTE: continuing loop
00118         }
00119         break;
00120
00121         case Choice::Invalid:
00122         {
00123             cout << "Usage: write C or F (or H for help)!\n";
00124         }
00125     }
00126 }
00127
00128 return result;
00129 }

```

6.2 hermann_serain.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef HERMANN_SERAIN_HPP
00002 #define HERMANN_SERAIN_HPP
00003
00004 #include <opencv2/opencv.hpp>
00005 #include <iostream>
00006 #include <vector>
00007 #include <map>
00008 #include <string>
00009 #include <numeric>
00010
00011 #include "shared.hpp"
00012 #include "marco_annunziata.hpp"
00013
00014 using namespace cv;
00015 using namespace std;
00016 using namespace Shared;
00017
00018 //NOTE: Contains the user choices
00019 struct UserData {
00020     Choice choice;
00021     string data_path;
00022
00023     UserData(Choice _choice, string _data_path)
00024         : choice(_choice), data_path(_data_path) {}
00025 };

```

```

00026
00027 struct MetricsCounter {
00028     int tp = 0; // True Positives
00029     int fp = 0; // False Positives
00030     int fn = 0; // False Negatives
00031 };
00032
00033 //NOTE: Public methods
00034 UserData readInput(int argc, char** argv);
00035 void computeImageMetrics(string imagePath, std::vector<Detection> predictedDetections);
00036
00037 #endif

```

6.3 include/marco_annunziata.hpp File Reference

```

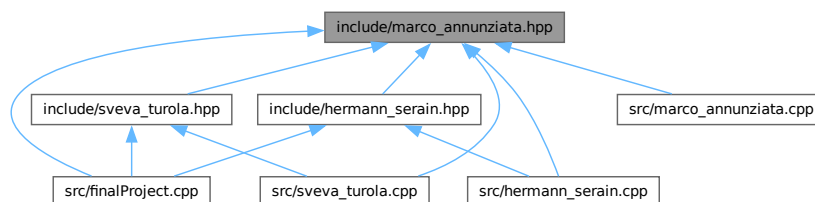
#include <opencv2/core/types.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/features2d.hpp>
#include <opencv2/dnn.hpp>
#include <onnxruntime_cxx_api.h>
#include "shared.hpp"
#include <iostream>
#include <streambuf>
#include <vector>
#include <fstream>

```

Include dependency graph for marco_annunziata.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Detection](#)
- struct [Detections](#)
- class [YOLO_model](#)

6.4 marco_annunziata.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef MARCO_ANNUNZIATA_HPP
00002 #define MARCO_ANNUNZIATA_HPP
00003
00004 #include <opencv2/core/types.hpp>
00005 #include <opencv2/highgui.hpp>
00006 #include <opencv2/imgproc.hpp>
00007 #include <opencv2/features2d.hpp>
00008 #include <opencv2/dnn.hpp>
00009 #include <onnxruntime_cxx_api.h>
00010 #include "shared.hpp"
00011 #include <iostream>
00012 #include <streambuf>
00013 #include <vector>
00014 #include <fstream>
00015
00016 struct Detection{
00017     cv::Rect boundingBox = cv::Rect(0,0,0,0);
00018     float classConfidence = 0;
00019     int classId = -1;
00020 };
00021
00022 struct Detections{
00023     std::vector<cv::Rect> boundingBoxes;
00024     std::vector<float> classConfidences;
00025     std::vector<int> classIds;
00026 };
00027
00028 class YOLO_model{
00029     private:
00030         Ort::Env env;
00031         static void logger(void* param, OrtLoggingLevel severity, const char*
category, const char* logid, const char* code_location, const char* message);
00032         Ort::Session session;
00033         Ort::SessionOptions sessionOptions;
00034         Ort::AllocatorWithDefaultOptions allocator;
00035         bool usingGPU = false;
00036         const int YOLO_TARGET_INPUT_SIZE = 640; // MUST be multiple of 32. See
YOLO_model::detectObjects implementation.
00037         std::vector<Detection> detections;
00038         std::string modelName = "Yolo";
00039         std::vector<std::string> classNames;
00040         const float CLASS_CONFIDENCE_THRESHOLD = 0.5;
00041         const float NMS_THRESHOLD = 0.5;
00042     public:
00043         YOLO_model();
00044         bool isAvailableGPU();
00045         std::vector<Detection> detectionPipeline(cv::Mat &img, bool enable_letterbox_padding=true);
00046         void clearDetections();
00047         Detections detect(const cv::Mat &img, bool enable_letterbox_padding=true);
00048         void mergeDetections(Detections& dest, const Detections& source);
00049         std::vector<std::string> getDataClasses(std::string
labelsFilename="../data/model/labels.txt");
00050         std::vector<Detection> filterDetectionsNMS(Detections goodDetections);
00051         cv::Mat drawBoundingBoxes(cv::Mat &img, std::vector<Detection> &detections,
cv::Scalar color);
00052         cv::Mat drawBoundingBoxes(cv::Mat &img, cv::Scalar color);
00053         void setModelName(std::string modelName);
00054         std::string getModelName();
00055         std::vector<Detection> getDetections();
00056         std::vector<std::string> getDetectionsClassNames(Detections detections);
00057         std::string getClassId(int classId);
00058     };
00059
00060 #endif

```

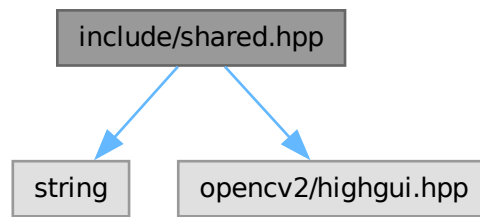
6.5 include/shared.hpp File Reference

```

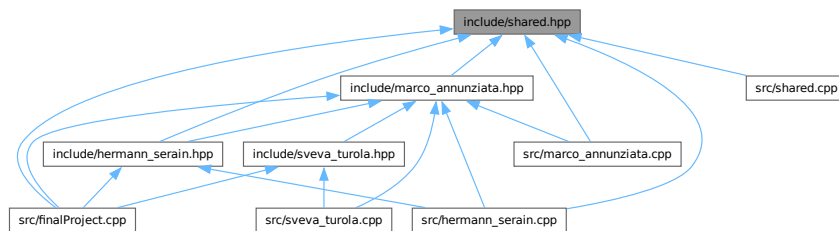
#include <string>
#include <opencv2/highgui.hpp>

```

Include dependency graph for shared.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [Shared](#)

Enumerations

- enum class [Shared::Choice](#) { [Shared::Camera](#) , [Shared::File](#) , [Shared::Invalid](#) , [Shared::Help](#) }

Functions

- [Choice Shared::parseChoice](#) (std::string s)

6.6 shared.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SHARED_HPP
00002 #define SHARED_HPP
00003
00004 #include <string>
00005 #include <opencv2/highgui.hpp>
00006
00007 namespace Shared{
00008
00009     //NOTE: Possible choices
00010     enum class Choice {Camera, File, Invalid, Help};
00011
00012     Choice parseChoice(std::string s);
00013 }
00014
00015 #endif
  
```



```

    Mat frame,
    int savedCount,
    string labels_path)

```

Definition at line 19 of file [sveva_turola.cpp](#).

```

00019                                     {
00020     YOLO_model model;
00021     model.setModelName("YOLO11s");
00022     vector<string> dataClasses = model.getDataClasses(labels_path);
00023     int frameCount = 0;
00024     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00025     int fps = cap.get(CAP_PROP_FPS);
00026     vector<Detection> detections;
00027
00028     cap.read(frame);
00029     if (frame.empty()){
00030         return -1;
00031     }
00032
00033     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00034     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00035     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00036     if(!out.isOpened()){
00037         cerr << "Could not open the output video file for write\n";
00038         return -1;
00039     }
00040
00041     cout << "Loading camera video..." << endl;
00042
00043     while (true) {
00044         cap.read(frame);
00045         if (frame.empty()){
00046             break;
00047         }
00048
00049         savedCount++;
00050         int key = waitKey(1) & 0xFF;
00051
00052         if (frameCount % (fps/2) == 0) { // TODO cambiare
00053             detections = model.detectionPipeline(frame);
00054         }
00055         frameCount++;
00056         Mat outputFrame = cardValues(detections, model, frame);
00057         out.write(outputFrame);
00058         namedWindow("Output Video", WINDOW_NORMAL);
00059         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00060         imshow("Output Video", outputFrame);
00061         waitKey(1);
00062
00063         if (key == 'q') {
00064             cout << "Closed camera!\n";
00065             break;
00066         }
00067     }
00068
00069     return savedCount;
00070 }

```

6.7.1.2 cardValues()

```

Mat cardValues (
    vector< Detection > detections,
    YOLO_model & model,
    Mat & frame)

```

Definition at line 155 of file [sveva_turola.cpp](#).

```

00155                                     {
00156     vector<Detection> green, blue, red;
00157
00158     for(auto detection : detections){
00159         char cardNumber = model.getClassName(detection.classId)[0];
00160
00161         if(cardNumber >= '2' && cardNumber <= '6'){
00162             green.push_back(detection);
00163         }
00164         else if(cardNumber >= '7' && cardNumber <= '9'){
00165             blue.push_back(detection);

```



```

00166         }
00167         else {
00168             red.push_back(detection);
00169         }
00170     }
00171
00172     Mat outputFrame = frame.clone();
00173
00174     if(!green.empty()){
00175         outputFrame = model.drawBoundingBoxes(outputFrame, green, Scalar(0, 255, 0));
00176     }
00177     if(!blue.empty()){
00178         outputFrame = model.drawBoundingBoxes(outputFrame, blue, Scalar(255, 0, 0));
00179     }
00180     if(!red.empty()){
00181         outputFrame = model.drawBoundingBoxes(outputFrame, red, Scalar(0, 0, 255));
00182     }
00183
00184     return outputFrame;
00185 }

```

6.7.1.3 frameCapture()

```

int frameCapture (
    string data_path,
    string labels_path)

```

Definition at line 119 of file [sveva_turola.cpp](#).

```

00119         {
00120             string outputDir = "../output/frames/"; //output directory
00121             Mat frame;
00122             int savedCount = 0;
00123             VideoCapture cap;
00124             VideoWriter out;
00125
00126             if (!fs::exists(outputDir)) { // creation of output directory
00127                 fs::create_directory(outputDir);
00128             }
00129
00130             if(data_path == "0"){
00131                 cap.open(0);
00132                 if (!cap.isOpened()) {
00133                     cerr << "Error opening camera!" << endl;
00134                     return -1;
00135                 }
00136                 savedCount = cameraCapture(cap, out, frame, savedCount, labels_path);
00137             } else {
00138                 cap.open(data_path);
00139                 if (!cap.isOpened()) {
00140                     cerr << "Error opening video!" << endl;
00141                     return -1;
00142                 }
00143                 savedCount = videoCapture(cap, out, frame, savedCount, labels_path);
00144             }
00145
00146             cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00147             cout << "Video saved in output directory" << endl;
00148
00149             cap.release();
00150             out.release();
00151             destroyAllWindows();
00152             return 0;
00153 }

```

6.7.1.4 videoCapture()

```

int videoCapture (
    VideoCapture cap,
    VideoWriter out,
    Mat frame,
    int savedCount,
    string labels_path)

```

Definition at line 72 of file [sveva_turola.cpp](#).

```

00072                                     {
00073     YOLO_model model;
00074     model.setModelName("YOLO11s");
00075     vector<string> dataClasses = model.getDataClasses(labels_path);
00076     int frameCount = 0;
00077     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00078     int fps = cap.get(CAP_PROP_FPS);
00079     vector<Detection> detections;
00080
00081     cap.read(frame);
00082     if (frame.empty()){
00083         return -1;
00084     }
00085
00086     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00087     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00088     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00089     if(!out.isOpened()){
00090         cerr << "Could not open the output video file for write\n";
00091         return -1;
00092     }
00093
00094     cout << "Loading video..." << endl;
00095
00096     while (true) {
00097         cap.read(frame);
00098         if (frame.empty()){
00099             break;
00100         }
00101
00102         savedCount++;
00103         int key = waitKey(1) & 0xFF;
00104
00105         if (frameCount % (fps/2) == 0) { //TODO cambiare
00106             detections = model.detectionPipeline(frame);
00107         }
00108         frameCount++;
00109         Mat outputFrame = cardValues(detections, model, frame);
00110         out.write(outputFrame);
00111         namedWindow("Output Video", WND_PROP_FULLSCREEN);
00112         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00113         imshow("Output Video", outputFrame);
00114         waitKey(1);
00115     }
00116     return savedCount;
00117 }

```

6.8 sveva_turola.hpp

[Go to the documentation of this file.](#)

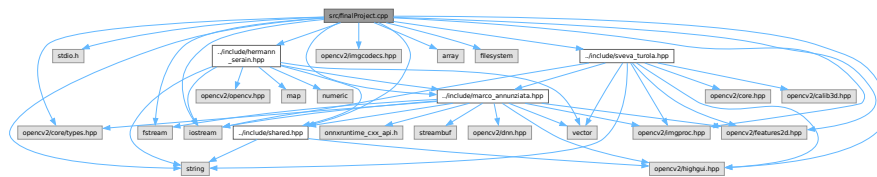
```

00001 #ifndef SVEVA_TUROLA_HPP
00002 #define SVEVA_TUROLA_HPP
00003
00004 #include <opencv2/features2d.hpp>
00005 #include <opencv2/highgui.hpp>
00006 #include <iostream>
00007 #include <string>
00008 #include <vector>
00009 #include <opencv2/core.hpp>
00010 #include <opencv2/imgproc.hpp>
00011 #include <opencv2/calib3d.hpp>
00012
00013 #include "marco_annunziata.hpp"
00014
00015 using namespace std;
00016 using namespace cv;
00017
00018 int videoCapture(VideoCapture cap, VideoWriter out, Mat frame, int savedCount, string labels_path);
00019 int cameraCapture(VideoCapture cap, VideoWriter out, Mat frame, int savedCount, string labels_path);
00020 int frameCapture(string data_path, string labels_path);
00021 Mat cardValues(vector<Detection> detections, YOLO_model &model, Mat &frame);
00022
00023 #endif

```

6.9 src/finalProject.cpp File Reference

```
#include <opencv2/core/types.hpp>
#include <stdio.h>
#include <iostream>
#include <string>
#include <fstream>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/imgcodecs.hpp>
#include <opencv2/features2d.hpp>
#include <array>
#include <filesystem>
#include "../include/shared.hpp"
#include "../include/marco_annunziata.hpp"
#include "../include/hermann_serain.hpp"
#include "../include/sveva_turola.hpp"
Include dependency graph for finalProject.cpp:
```



Functions

- `int main (int argc, char **argv)`

6.9.1 Function Documentation

6.9.1.1 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 24 of file `finalProject.cpp`.

```
00024     {
00025         string choice = "X";
00026         string data_path;
00027         string labels_path = "../data/model/labels.txt";
00028         enum class fileCategories {IMAGE, VIDEO, UNKNOWN};
00029         auto fileType = fileCategories::UNKNOWN;
00030         string allowedImgType[] = {".png", ".jpg", ".jpeg"};
00031         string allowedVidType[] = {".mp4"};
00032         YOLO_model model;
00033         model.setModelName("YOLO11s");
00034         UserData userData = readInput(argc, argv);
00035
00036         switch (userData.choice)
00037         {
00038             case Choice::Camera:
00039             {
00040                 frameCapture("0", labels_path);
00041             }
```

```

00042         break;
00043
00044         case Choice::File:
00045         {
00046             /* check if file is image */
00047             for(auto type : allowedImgType){
00048                 if(userData.data_path.rfind(type) != string::npos){
00049                     fileType = fileCategories::IMAGE;
00050                     Mat frame = imread(userData.data_path, cv::IMREAD_COLOR);
00051
00052                     vector<string> dataClasses = model.getDataClasses(labels_path);
00053                     cout << "GPU available? " << (model.isAvailableGPU() ? "YES" : "NO") << endl;
00054                     model.detectionPipeline(frame);
00055                     Mat resultImg = model.drawBoundingBoxes(frame, Scalar(255, 0, 0));
00056                     auto detections = model.getDetections();
00057                     computeImageMetrics(userData.data_path, detections);
00058                     std::string windowTitle = model.getModelName() + " - " +
std::to_string(detections.size()) + " detections";
00059                     namedWindow(windowTitle, WINDOW_NORMAL);
00060                     imshow(windowTitle, resultImg);
00061                     waitKey(0);
00062                     break;
00063                 }
00064             }
00065
00066             /* check if file is video */
00067             if(fileType == fileCategories::UNKNOWN){
00068                 for(auto type: allowedVidType){
00069                     if(userData.data_path.rfind(type) != string::npos){
00070                         fileType = fileCategories::VIDEO;
00071                         cout << "Using ONNX Runtime GPU? " << (model.isAvailableGPU() ? "YES" : "NO") <<
endl;
00072                         frameCapture(userData.data_path, labels_path);
00073                         break;
00074                     }
00075                 }
00076             }
00077
00078             /* invalid file */
00079             if(fileType == fileCategories::UNKNOWN){
00080                 cerr << "The image or video provided has an invalid file type. Please retry with a
different one.\n";
00081                 return -1;
00082             }
00083
00084             }
00085         }
00086         break;
00087
00088         case Choice::Invalid:
00089         case Choice::Help:
00090         {
00091             //TODO: Verifica se questa parte va bene, di norma in questo caso non può ricaderci
l'utente
00092             throw std::logic_error("Invalid operation: ...");
00093         }
00094     }
00095
00096     // TODO: batch processing
00097     if(labels_path.rfind(".txt") == string::npos){
00098         cerr << "The label file provided has an invalid file type. Please provide a .txt file.\n";
00099         return -1;
00100     }
00101
00102     // destroyAllWindows();
00103     return 0;
00104 }

```

6.10 finalProject.cpp

[Go to the documentation of this file.](#)

```

00001 //includes
00002 #include <opencv2/core/types.hpp>
00003 #include <stdio.h>
00004 #include <iostream>
00005 #include <string>
00006 #include <fstream>
00007 #include <opencv2/highgui.hpp>
00008 #include <opencv2/imgproc.hpp>
00009 #include <opencv2/imgcodecs.hpp>
00010 #include <opencv2/features2d.hpp>

```

```

00011 #include <array>
00012 #include <filesystem>
00013
00014 //libs
00015 #include "../include/shared.hpp"
00016 #include "../include/marco_annunziata.hpp"
00017 #include "../include/hermann_serain.hpp"
00018 #include "../include/sveva_turola.hpp"
00019
00020 using namespace std;
00021 using namespace cv;
00022 using namespace Shared;
00023
00024 int main(int argc, char** argv){
00025     string choice = "X";
00026     string data_path;
00027     string labels_path = "../data/model/labels.txt";
00028     enum class fileCategories {IMAGE, VIDEO, UNKNOWN};
00029     auto fileType = fileCategories::UNKNOWN;
00030     string allowedImgType[] = {".png", ".jpg", ".jpeg"};
00031     string allowedVidType[] = {".mp4"};
00032     YOLO_model model;
00033     model.setModelName("YOLO11s");
00034     UserData userData = readInput(argc, argv);
00035
00036     switch (userData.choice)
00037     {
00038     case Choice::Camera:
00039     {
00040         frameCapture("0", labels_path);
00041     }
00042     break;
00043
00044     case Choice::File:
00045     {
00046         /* check if file is image */
00047         for(auto type : allowedImgType){
00048             if(userData.data_path.rfind(type) != string::npos){
00049                 fileType = fileCategories::IMAGE;
00050                 Mat frame = imread(userData.data_path, cv::IMREAD_COLOR);
00051
00052                 vector<string> dataClasses = model.getDataClasses(labels_path);
00053                 cout << "GPU available? " << (model.isAvailableGPU() ? "YES" : "NO") << endl;
00054                 model.detectionPipeline(frame);
00055                 Mat resultImg = model.drawBoundingBoxes(frame, Scalar(255, 0, 0));
00056                 auto detections = model.getDetections();
00057                 computeImageMetrics(userData.data_path, detections);
00058                 std::string windowTitle = model.getModelName() + " - " +
std::to_string(detections.size()) + " detections";
00059                 namedWindow(windowTitle, WINDOW_NORMAL);
00060                 imshow(windowTitle, resultImg);
00061                 waitKey(0);
00062                 break;
00063             }
00064         }
00065
00066         /* check if file is video */
00067         if(fileType == fileCategories::UNKNOWN){
00068             for(auto type: allowedVidType){
00069                 if(userData.data_path.rfind(type) != string::npos){
00070                     fileType = fileCategories::VIDEO;
00071                     cout << "Using ONNX Runtime GPU? " << (model.isAvailableGPU() ? "YES" : "NO") <<
endl;
00072                     frameCapture(userData.data_path, labels_path);
00073                     break;
00074                 }
00075             }
00076         }
00077
00078         /* invalid file */
00079         if(fileType == fileCategories::UNKNOWN){
00080             cerr << "The image or video provided has an invalid file type. Please retry with a
different one.\n";
00081             return -1;
00082         }
00083
00084     }
00085     break;
00086
00087     case Choice::Invalid:
00088     case Choice::Help:
00089     {
00090         //TODO: Verifica se questa parte va bene, di norma in questo caso non può ricaderci
l'utente
00092         throw std::logic_error("Invalid operation: ...");
00093     }

```

```

00094     }
00095
00096     // TODO: batch processing
00097     if(labels_path.rfind(".txt") == string::npos){
00098         cerr << "The label file provided has an invalid file type. Please provide a .txt file.\n";
00099         return -1;
00100     }
00101
00102     // destroyAllWindows();
00103     return 0;
00104 }

```

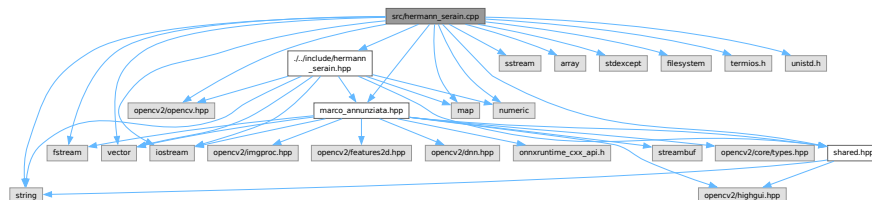
6.11 src/hermann_serain.cpp File Reference

```

#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <numeric>
#include <fstream>
#include <sstream>
#include <array>
#include <stdexcept>
#include <filesystem>
#include <termios.h>
#include <unistd.h>
#include "../include/hermann_serain.hpp"
#include "../include/marco_annunziata.hpp"
#include "../include/shared.hpp"

```

Include dependency graph for hermann_serain.cpp:



Classes

- struct [TerminalRawMode](#)

Functions

- void [splitPath](#) (const string &fullPath, string &directory, string &partial)
- vector< string > [findMatches](#) (const string &directory, const string &partial)
- void [redrawLine](#) (const string &text)
- void [printHelp](#) ()
- void [handleCamera](#) ()
- string [handleFile](#) ()
- [UserData](#) [readInput](#) (int argc, char **argv)
- bool [checkPath](#) (const string &fullPath)

- bool [canComputeMetrics](#) (string filePath, string directoryName)
- bool [canComputeImageMetrics](#) (string imgPath)
- bool [canComputeVideoMetrics](#) (string videoPath)
- float [calculateIoU](#) (const cv::Rect &box1, const cv::Rect &box2)
- [MetricsCounter](#) [updateMetricsCounters](#) (std::vector< [Detection](#) > predictedDetections, std::vector< [Detection](#) > actualDetections)
- void [printFinalMetrics](#) (const [MetricsCounter](#) &total)
- std::vector< [Detection](#) > [parseYoloLabels](#) (string labelPath, int imgW, int imgH)
- void [computeImageMetrics](#) (string imagePath, std::vector< [Detection](#) > predictedDetections)

Variables

- const std::string [DEFAULT_VIDEO_PATH](#) = "../data/test/videos/default_video.mp4"
- const std::string [DEFAULT_IMAGE_PATH](#) = "../data/test/images/default_image.jpg"
- const std::string [FLAG_VIDEO](#) = "--default-video"
- const std::string [FLAG_IMAGE](#) = "--default-image"

6.11.1 Function Documentation

6.11.1.1 [calculateIoU\(\)](#)

```
float calculateIoU (
    const cv::Rect & box1,
    const cv::Rect & box2)
```

Definition at line 304 of file [hermann_serain.cpp](#).

```
00304                                     {
00305     int x1 = std::max(box1.x, box2.x);
00306     int y1 = std::max(box1.y, box2.y);
00307     int x2 = std::min(box1.x + box1.width, box2.x + box2.width);
00308     int y2 = std::min(box1.y + box1.height, box2.y + box2.height);
00309
00310     int intersectionArea = std::max(0, x2 - x1) * std::max(0, y2 - y1);
00311     int unionArea = box1.area() + box2.area() - intersectionArea;
00312
00313     if (unionArea == 0) return 0.0f;
00314     return static_cast<float>(intersectionArea) / unionArea;
00315 }
```

6.11.1.2 [canComputeImageMetrics\(\)](#)

```
bool canComputeImageMetrics (
    string imgPath)
```

Definition at line 295 of file [hermann_serain.cpp](#).

```
00295                                     {
00296     return canComputeMetrics(imgPath, "images");
00297 }
```

6.11.1.3 canComputeMetrics()

```
bool canComputeMetrics (
    string filePath,
    string directoryName)
```

Definition at line 279 of file [hermann_serain.cpp](#).

```
00279                                     {
00280     fs::path p(filePath);
00281     string filename = p.stem().string();
00282
00283     // Construct the path dynamically based on directoryName
00284     string labelPath = "../data/test/ground_truths/" + directoryName + "/" + filename + ".txt";
00285
00286     if (checkPath(labelPath)) {
00287         cout << "[METRICS] Ground truth found in " << directoryName << " for: " << filename << ".
Proceeding with calculation." << endl;
00288         return true;
00289     }
00290
00291     cout << "[INFO] No ground truth labels found at: " << labelPath << ". Skipping metrics." << endl;
00292     return false;
00293 }
```

6.11.1.4 canComputeVideoMetrics()

```
bool canComputeVideoMetrics (
    string videoPath)
```

Definition at line 299 of file [hermann_serain.cpp](#).

```
00299                                     {
00300     return canComputeMetrics(videoPath, "videos");
00301 }
```

6.11.1.5 checkPath()

```
bool checkPath (
    const string & fullPath)
```

Definition at line 268 of file [hermann_serain.cpp](#).

```
00268                                     {
00269     try {
00270         if (fs::exists(fullPath) && fs::is_regular_file(fullPath)) {
00271             return true;
00272         }
00273     } catch (const fs::filesystem_error& e) {
00274         cerr << "[ERROR] Filesystem access error: " << e.what() << endl;
00275     }
00276     return false;
00277 }
```

6.11.1.6 computeImageMetrics()

```
void computeImageMetrics (
    string imagePath,
    std::vector< Detection > predictedDetections)
```

Definition at line 438 of file [hermann_serain.cpp](#).

```
00438                                     {
00439     if (!canComputeImageMetrics(imagePath)) {
00440         return;
00441     }
00442 }
```



```

00443     fs::path p(imagePath);
00444     string filename = p.stem().string();
00445     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00446
00447     cv::Mat img = cv::imread(imagePath);
00448     if (img.empty()) {
00449         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00450         return;
00451     }
00452
00453     // Extraction of ground truth using the helper method
00454     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00455
00456     if (!actualDetections.empty()) {
00457         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00458         printFinalMetrics(results);
00459     }
00460 }

```

6.11.1.7 findMatches()

```

vector< string > findMatches (
    const string & directory,
    const string & partial)

```

Definition at line 242 of file [hermann_serain.cpp](#).

```

00243 {
00244     vector<string> matches;
00245
00246     if (!fs::exists(directory))
00247         return matches;
00248
00249     for (const auto& entry : fs::directory_iterator(directory))
00250     {
00251         string name = entry.path().filename().string();
00252
00253         if (name.rfind(partial, 0) == 0)
00254             matches.push_back(name);
00255     }
00256
00257     return matches;
00258 }

```

6.11.1.8 handleCamera()

```

void handleCamera ()

```

Definition at line 154 of file [hermann_serain.cpp](#).

```

00155 {
00156     cout << "Selected camera (press q to close the camera)\n";
00157 }

```

6.11.1.9 handleFile()

```

string handleFile ()

```

Definition at line 159 of file [hermann_serain.cpp](#).

```

00160 {
00161     TerminalRawMode rawMode; // abilita modalità raw
00162
00163     string currentPath;
00164     cout << "Insert file path: ";
00165     cout.flush();
00166
00167     while (true)
00168     {
00169         char ch;
00170         read(STDIN_FILENO, &ch, 1);

```

```

00171
00172 // ENTER
00173 if (ch == '\n')
00174 {
00175     cout << endl;
00176     return currentPath;
00177 }
00178
00179 // BACKSPACE
00180 if (ch == 127)
00181 {
00182     if (!currentPath.empty())
00183     {
00184         currentPath.pop_back();
00185         redrawLine(currentPath);
00186     }
00187     continue;
00188 }
00189
00190 // TAB
00191 if (ch == '\t')
00192 {
00193     string dir, partial;
00194     splitPath(currentPath, dir, partial);
00195
00196     auto matches = findMatches(dir, partial);
00197
00198     if (matches.size() == 1)
00199     {
00200         currentPath.erase(currentPath.size() - partial.size());
00201         currentPath += matches[0];
00202     }
00203     else if (matches.size() > 1)
00204     {
00205         cout << "\n";
00206         for (const auto& m : matches)
00207             cout << " " << m << "\n";
00208     }
00209
00210     redrawLine(currentPath);
00211     continue;
00212 }
00213
00214 // Normal char
00215 if (isprint(ch))
00216 {
00217     currentPath.push_back(ch);
00218     cout << ch;
00219     cout.flush();
00220 }
00221 }
00222 }

```

6.11.1.10 parseYoloLabels()

```

std::vector< Detection > parseYoloLabels (
    string labelPath,
    int imgW,
    int imgH)

```

Definition at line 398 of file [hermann_serain.cpp](#).

```

00398
00399 std::vector<Detection> actualDetections;
00400 ifstream file(labelPath);
00401 string line;
00402
00403 try {
00404     while (getline(file, line)) {
00405         if (line.empty()) continue;
00406
00407         stringstream ss(line);
00408         int classId;
00409         float x_center, y_center, w, h;
00410
00411         if (!(ss >> classId >> x_center >> y_center >> w >> h)) {
00412             throw std::runtime_error("Invalid YOLO format in file: " + labelPath);
00413         }
00414
00415         //NOTE: Converting bounding box width and height from percentage to actual values
00416         int pixelW = static_cast<int>(w * imgW);

```

```

00417         int pixelH = static_cast<int>(h * imgH);
00418         //NOTE: Moving the bounding box center to the left top corner.
00419         int pixelX = static_cast<int>((x_center * imgW) - (pixelW / 2.0f));
00420         int pixelY = static_cast<int>((y_center * imgH) - (pixelH / 2.0f));
00421
00422         Detection gt;
00423         gt.classId = classId;
00424         gt.boundingBox = cv::Rect(pixelX, pixelY, pixelW, pixelH);
00425         gt.classConfidence = 1.0f;
00426
00427         actualDetections.push_back(gt);
00428     }
00429     } catch (const std::exception& e) {
00430         cerr << "[ERROR] Reading image ground truth: " << e.what() << endl;
00431         file.close();
00432         return {}; // Return empty vector on error
00433     }
00434     file.close();
00435     return actualDetections;
00436 }

```

6.11.1.11 printFinalMetrics()

```

void printFinalMetrics (
    const MetricsCounter & total)

```

Definition at line 377 of file [hermann_serain.cpp](#).

```

00377     {
00378         if (total.tp + total.fp == 0 || total.tp + total.fn == 0) {
00379             cout << "[METRICS] Warning: Not enough data to compute Precision/Recall." << endl;
00380             return;
00381         }
00382
00383         float precision = static_cast<float>(total.tp) / (total.tp + total.fp);
00384         float recall = static_cast<float>(total.tp) / (total.tp + total.fn);
00385         float f1Score = (precision + recall > 0) ? (2.0f * (precision * recall) / (precision + recall)) :
00386             0.0f;
00387         cout << "\n--- Final Performance Metrics ---" << endl;
00388         cout << "Total True Positives (TP): " << total.tp << endl;
00389         cout << "Total False Positives (FP): " << total.fp << endl;
00390         cout << "Total False Negatives (FN): " << total.fn << endl;
00391         cout << "-----" << endl;
00392         cout << "Precision: " << precision << endl;
00393         cout << "Recall: " << recall << endl;
00394         cout << "F1-Score: " << f1Score << endl;
00395         cout << "-----\n" << endl;
00396     }

```

6.11.1.12 printHelp()

```

void printHelp ()

```

Definition at line 133 of file [hermann_serain.cpp](#).

```

00134 {
00135     cout << "\n";
00136     cout << "===== \n";
00137     cout << "          APPLICATION HELP PAGE          \n";
00138     cout << "===== \n";
00139     cout << "\n";
00140     cout << "Commands: \n";
00141     cout << "  C          Use camera as input \n";
00142     cout << "  F          Use file as input \n";
00143     cout << "  H / -h     Show this help page \n";
00144     cout << "\n";
00145     cout << "Details: \n";
00146     cout << "  - Camera closes by pressing 'q' \n";
00147     cout << "  - File path must not contain spaces \n";
00148     cout << "  - TAB completion not implemented yet \n";
00149     cout << "\n";
00150     cout << "===== \n";
00151     cout << endl;
00152 }

```

6.11.1.13 readInput()

```
UserData readInput (
    int argc,
    char ** argv)
```

Definition at line 63 of file [hermann_serain.cpp](#).

```
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG_VIDEO) {
00073                 result.data_path = DEFAULT_VIDEO_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG_IMAGE) {
00077                 result.data_path = DEFAULT_IMAGE_PATH;
00078                 result.choice = Choice::File;
00079             }
00080         }
00081     }
00082
00083     if(result.choice != Choice::Invalid)
00084         return result;
00085
00086     //NOTE: I'm gonna read the user input from now on
00087     string choiceStr;
00088
00089     // NOTE: Let's keep asking if the input is invalid
00090     while (result.choice == Choice::Invalid)
00091     {
00092         cout << "-----\n";
00093         cout << " C = Camera | F = File | H = Help\n";
00094         cout << "-----\n";
00095         cout << "Write your choice: ";
00096
00097         cin >> choiceStr;
00098         result.choice = parseChoice(choiceStr);
00099
00100         switch (result.choice)
00101         {
00102             case Choice::Camera:
00103             {
00104                 handleCamera();
00105             }
00106             break;
00107
00108             case Choice::File:
00109             {
00110                 result.data_path = handleFile();
00111             }
00112             break;
00113
00114             case Choice::Help:
00115             {
00116                 printHelp();
00117                 result.choice = Choice::Invalid; //NOTE: continuing loop
00118             }
00119             break;
00120
00121             case Choice::Invalid:
00122             {
00123                 cout << "Usage: write C or F (or H for help)!\n";
00124             }
00125         }
00126     }
00127
00128     return result;
00129 }
```

6.11.1.14 redrawLine()

```
void redrawLine (
    const string & text)
```

Definition at line 260 of file [hermann_serain.cpp](#).

```
00261 {
00262     cout << "\rInsert file path: " << text;
00263     cout << "\033[K"; // delete at the row end
00264     cout.flush();
00265 }
```

6.11.1.15 splitPath()

```
void splitPath (
    const string & fullPath,
    string & directory,
    string & partial)
```

Definition at line 226 of file [hermann_serain.cpp](#).

```
00227 {
00228     size_t pos = fullPath.find_last_of('/');
00229
00230     if (pos == string::npos)
00231     {
00232         directory = ".";
00233         partial = fullPath;
00234     }
00235     else
00236     {
00237         directory = fullPath.substr(0, pos);
00238         partial = fullPath.substr(pos + 1);
00239     }
00240 }
```

6.11.1.16 updateMetricsCounters()

```
MetricsCounter updateMetricsCounters (
    std::vector< Detection > predictedDetections,
    std::vector< Detection > actualDetections)
```

Definition at line 318 of file [hermann_serain.cpp](#).

```
00318 {
00319     const float IOU_THRESHOLD = 0.5f;
00320     MetricsCounter frameResults;
00321
00322     // Copy of ground truth to manage matching and removals
00323     std::vector<Detection> remainingGT = actualDetections;
00324
00325     for (const auto& pred : predictedDetections) {
00326         int bestMatchIdx = -1;
00327         float maxIoU = 0.0f;
00328
00329         // Search for the best Ground Truth match (Same Class + Highest IoU)
00330         for (int i = 0; i < remainingGT.size(); ++i) {
00331             if (pred.classId == remainingGT[i].classId) {
00332                 float currentIoU = calculateIoU(pred.boundingBox, remainingGT[i].boundingBox);
00333                 if (currentIoU > maxIoU) {
00334                     maxIoU = currentIoU;
00335                     bestMatchIdx = i;
00336                 }
00337             }
00338         }
00339
00340         // Validate the match against the threshold
00341         if (bestMatchIdx != -1 && maxIoU >= IOU_THRESHOLD) {
00342             frameResults.tp++;
00343
00344             //WARNING: CHECK CORRECTNESS WITH DEBUGING
00345
00346             /* CARD CONSOLIDATION LOGIC
00347              Per your requirement: remove the matched GT and other GTs of the
00348              same class within the surrounding "card-sized" region.
00349             */
00350             cv::Rect matchedRect = remainingGT[bestMatchIdx].boundingBox;
00351             int matchedClass = remainingGT[bestMatchIdx].classId;
```

```

00352
00353         // Remove the exact match first
00354         remainingGT.erase(remainingGT.begin() + bestMatchIdx);
00355
00356         // Remove nearby boxes of the same card (approx. same region)
00357         remainingGT.erase(
00358             std::remove_if(remainingGT.begin(), remainingGT.end(),
00359                 [&](const Detection& d) {
00360                     return (d.classId == matchedClass && calculateIoU(matchedRect, d.boundingBox)
00361 > 0.1f);
00362                 })),
00363             remainingGT.end()
00364         );
00365     } else {
00366         frameResults.fp++; //NOTE: No valid ground truths found for this prediction
00367     }
00368 }
00369
00370 // NOTE: Ricordati che marco ha etichettato il video in modo strano !
00371 // Remains ground truths are false negative
00372 frameResults.fn = static_cast<int>(remainingGT.size());
00373
00374 return frameResults;
00375 }

```

6.11.2 Variable Documentation

6.11.2.1 DEFAULT_IMAGE_PATH

```
const std::string DEFAULT_IMAGE_PATH = "../data/test/images/default_image.jpg"
```

Definition at line 26 of file [hermann_serain.cpp](#).

6.11.2.2 DEFAULT_VIDEO_PATH

```
const std::string DEFAULT_VIDEO_PATH = "../data/test/videos/default_video.mp4"
```

Definition at line 25 of file [hermann_serain.cpp](#).

6.11.2.3 FLAG_IMAGE

```
const std::string FLAG_IMAGE = "--default-image"
```

Definition at line 28 of file [hermann_serain.cpp](#).

6.11.2.4 FLAG_VIDEO

```
const std::string FLAG_VIDEO = "--default-video"
```

Definition at line 27 of file [hermann_serain.cpp](#).

6.12 hermann_serain.cpp

[Go to the documentation of this file.](#)

```

00001 #include <opencv2/opencv.hpp>
00002 #include <iostream>
00003 #include <vector>
00004 #include <map>
00005 #include <string>
00006 #include <numeric>
00007 #include <fstream>
00008 #include <sstream>
00009 #include <array>
00010 #include <stdexcept>
00011 #include <filesystem>
00012 #include <termios.h>
00013 #include <unistd.h>
00014
00015 #include "../include/hermann_serain.hpp"
00016 #include "../include/marco_annunziata.hpp"
00017 #include "../include/shared.hpp"
00018
00019 namespace fs = std::filesystem;
00020 using namespace cv;
00021 using namespace std;
00022 using namespace Shared;
00023
00024 //User parameters consts
00025 const std::string DEFAULT_VIDEO_PATH = "../data/test/videos/default_video.mp4";
00026 const std::string DEFAULT_IMAGE_PATH = "../data/test/images/default_image.jpg";
00027 const std::string FLAG_VIDEO = "--default-video";
00028 const std::string FLAG_IMAGE = "--default-image";
00029
00030 //Tab utilities
00031 struct TerminalRawMode
00032 {
00033     termios oldt;
00034
00035     TerminalRawMode()
00036     {
00037         tcgetattr(STDIN_FILENO, &oldt);
00038         termios newt = oldt;
00039
00040         newt.c_lflag &= ~(ICANON | ECHO); // no buffer, no echo
00041         tcsetattr(STDIN_FILENO, TCSANOW, &newt);
00042     }
00043
00044     ~TerminalRawMode()
00045     {
00046         tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
00047     }
00048 };
00049
00050 void splitPath(const string& fullPath, string& directory, string& partial);
00051 vector<string> findMatches(const string& directory, const string& partial);
00052 void redrawLine(const string& text);
00053
00054
00055 //Help page methods
00056 void printHelp();
00057 void handleCamera();
00058 string handleFile();
00059
00060
00061 //WARNING: THIS PROGRAM USE THE TERMINAL API AND PROCESS THE TERMINAL IN RAW MODE,
00062 //IF THE TERMINAL DOESN'T WORK AS BEFORE TYPE BLINDED "reset"
00063 UserData readInput(int argc, char** argv)
00064 {
00065     //NOTE: Here I'm gonna read the user input from the terminal
00066     UserData result(Choice::Invalid, "");
00067
00068     if(argc > 0) {
00069         std::vector<std::string> args(argv, argv + argc);
00070
00071         for (int i = 1; i < argc; ++i) {
00072             if (args[i] == FLAG_VIDEO) {
00073                 result.data_path = DEFAULT_VIDEO_PATH;
00074                 result.choice = Choice::File;
00075             }
00076             else if (args[i] == FLAG_IMAGE) {
00077                 result.data_path = DEFAULT_IMAGE_PATH;
00078                 result.choice = Choice::File;
00079             }
00080         }
00081     }
00082

```

```

00083     if(result.choice != Choice::Invalid)
00084         return result;
00085
00086     //NOTE: I'm gonna read the user input from now on
00087     string choiceStr;
00088
00089     // NOTE: Let's keep asking if the input is invalid
00090     while (result.choice == Choice::Invalid)
00091     {
00092         cout << "-----\n";
00093         cout << " C = Camera | F = File | H = Help\n";
00094         cout << "-----\n";
00095         cout << "Write your choice: ";
00096
00097         cin >> choiceStr;
00098         result.choice = parseChoice(choiceStr);
00099
00100         switch (result.choice)
00101         {
00102             case Choice::Camera:
00103             {
00104                 handleCamera();
00105             }
00106             break;
00107
00108             case Choice::File:
00109             {
00110                 result.data_path = handleFile();
00111             }
00112             break;
00113
00114             case Choice::Help:
00115             {
00116                 printHelp();
00117                 result.choice = Choice::Invalid; //NOTE: continuing loop
00118             }
00119             break;
00120
00121             case Choice::Invalid:
00122             {
00123                 cout << "Usage: write C or F (or H for help)!\n";
00124             }
00125         }
00126     }
00127
00128     return result;
00129 }
00130
00131
00132 //NOTE: Print the help page
00133 void printHelp()
00134 {
00135     cout << "\n";
00136     cout << "===== \n";
00137     cout << "          APPLICATION HELP PAGE          \n";
00138     cout << "===== \n";
00139     cout << "\n";
00140     cout << "Commands:\n";
00141     cout << "  C          Use camera as input\n";
00142     cout << "  F          Use file as input\n";
00143     cout << "  H / -h     Show this help page\n";
00144     cout << "\n";
00145     cout << "Details:\n";
00146     cout << "  - Camera closes by pressing 'q'\n";
00147     cout << "  - File path must not contain spaces\n";
00148     cout << "  - TAB completion not implemented yet\n";
00149     cout << "\n";
00150     cout << "===== \n";
00151     cout << endl;
00152 }
00153
00154 void handleCamera()
00155 {
00156     cout << "Selected camera (press q to close the camera)\n";
00157 }
00158
00159 string handleFile()
00160 {
00161     TerminalRawMode rawMode; // abilita modalità raw
00162
00163     string currentPath;
00164     cout << "Insert file path: ";
00165     cout.flush();
00166
00167     while (true)
00168     {
00169         char ch;

```



```

00170         read(STDIN_FILENO, &ch, 1);
00171
00172         // ENTER
00173         if (ch == '\n')
00174         {
00175             cout << endl;
00176             return currentPath;
00177         }
00178
00179         // BACKSPACE
00180         if (ch == 127)
00181         {
00182             if (!currentPath.empty())
00183             {
00184                 currentPath.pop_back();
00185                 redrawLine(currentPath);
00186             }
00187             continue;
00188         }
00189
00190         // TAB
00191         if (ch == '\t')
00192         {
00193             string dir, partial;
00194             splitPath(currentPath, dir, partial);
00195
00196             auto matches = findMatches(dir, partial);
00197
00198             if (matches.size() == 1)
00199             {
00200                 currentPath.erase(currentPath.size() - partial.size());
00201                 currentPath += matches[0];
00202             }
00203             else if (matches.size() > 1)
00204             {
00205                 cout << "\n";
00206                 for (const auto& m : matches)
00207                     cout << " " << m << "\n";
00208             }
00209
00210             redrawLine(currentPath);
00211             continue;
00212         }
00213
00214         // Normal char
00215         if (isprint(ch))
00216         {
00217             currentPath.push_back(ch);
00218             cout << ch;
00219             cout.flush();
00220         }
00221     }
00222 }
00223
00224
00225 //Print utilities implementations
00226 void splitPath(const string& fullPath, string& directory, string& partial)
00227 {
00228     size_t pos = fullPath.find_last_of('/');
00229
00230     if (pos == string::npos)
00231     {
00232         directory = ".";
00233         partial = fullPath;
00234     }
00235     else
00236     {
00237         directory = fullPath.substr(0, pos);
00238         partial = fullPath.substr(pos + 1);
00239     }
00240 }
00241
00242 vector<string> findMatches(const string& directory, const string& partial)
00243 {
00244     vector<string> matches;
00245
00246     if (!fs::exists(directory))
00247         return matches;
00248
00249     for (const auto& entry : fs::directory_iterator(directory))
00250     {
00251         string name = entry.path().filename().string();
00252
00253         if (name.rfind(partial, 0) == 0)
00254             matches.push_back(name);
00255     }
00256 }

```

```

00257     return matches;
00258 }
00259
00260 void redrawLine(const string& text)
00261 {
00262     cout << "\rInsert file path: " << text;
00263     cout << "\033[K"; // delete at the row end
00264     cout.flush();
00265 }
00266
00267 //NOTE: Metrics
00268 bool checkPath(const string& fullPath) {
00269     try {
00270         if (fs::exists(fullPath) && fs::is_regular_file(fullPath)) {
00271             return true;
00272         }
00273     } catch (const fs::filesystem_error& e) {
00274         cerr << "[ERROR] Filesystem access error: " << e.what() << endl;
00275     }
00276     return false;
00277 }
00278
00279 bool canComputeMetrics(string filePath, string directoryName) {
00280     fs::path p(filePath);
00281     string filename = p.stem().string();
00282
00283     // Construct the path dynamically based on directoryName
00284     string labelPath = "../data/test/ground_truths/" + directoryName + "/" + filename + ".txt";
00285
00286     if (checkPath(labelPath)) {
00287         cout << "[METRICS] Ground truth found in " << directoryName << " for: " << filename << ".
Proceeding with calculation." << endl;
00288         return true;
00289     }
00290
00291     cout << "[INFO] No ground truth labels found at: " << labelPath << ". Skipping metrics." << endl;
00292     return false;
00293 }
00294
00295 bool canComputeImageMetrics(string imgPath) {
00296     return canComputeMetrics(imgPath, "images");
00297 }
00298
00299 bool canComputeVideoMetrics(string videoPath) {
00300     return canComputeMetrics(videoPath, "videos");
00301 }
00302
00303 //NOTE: Helper method to compute the IoU
00304 float calculateIoU(const cv::Rect& box1, const cv::Rect& box2) {
00305     int x1 = std::max(box1.x, box2.x);
00306     int y1 = std::max(box1.y, box2.y);
00307     int x2 = std::min(box1.x + box1.width, box2.x + box2.width);
00308     int y2 = std::min(box1.y + box1.height, box2.y + box2.height);
00309
00310     int intersectionArea = std::max(0, x2 - x1) * std::max(0, y2 - y1);
00311     int unionArea = box1.area() + box2.area() - intersectionArea;
00312
00313     if (unionArea == 0) return 0.0f;
00314     return static_cast<float>(intersectionArea) / unionArea;
00315 }
00316
00317 //NOTE: Method used to compute the Detections for both images and videos
00318 MetricsCounter updateMetricsCounters(std::vector<Detection> predictedDetections,
std::vector<Detection> actualDetections) {
00319     const float IOU_THRESHOLD = 0.5f;
00320     MetricsCounter frameResults;
00321
00322     // Copy of ground truth to manage matching and removals
00323     std::vector<Detection> remainingGT = actualDetections;
00324
00325     for (const auto& pred : predictedDetections) {
00326         int bestMatchIdx = -1;
00327         float maxIoU = 0.0f;
00328
00329         // Search for the best Ground Truth match (Same Class + Highest IoU)
00330         for (int i = 0; i < remainingGT.size(); ++i) {
00331             if (pred.classId == remainingGT[i].classId) {
00332                 float currentIoU = calculateIoU(pred.boundingBox, remainingGT[i].boundingBox);
00333                 if (currentIoU > maxIoU) {
00334                     maxIoU = currentIoU;
00335                     bestMatchIdx = i;
00336                 }
00337             }
00338         }
00339
00340         // Validate the match against the threshold
00341         if (bestMatchIdx != -1 && maxIoU >= IOU_THRESHOLD) {

```

```

00342         frameResults.tp++;
00343
00344         //WARNING: CHECK CORRECTNESS WITH DEBUGING
00345
00346         /* CARD CONSOLIDATION LOGIC
00347         Per your requirement: remove the matched GT and other GTs of the
00348         same class within the surrounding "card-sized" region.
00349         */
00350         cv::Rect matchedRect = remainingGT[bestMatchIdx].boundingBox;
00351         int matchedClass = remainingGT[bestMatchIdx].classId;
00352
00353         // Remove the exact match first
00354         remainingGT.erase(remainingGT.begin() + bestMatchIdx);
00355
00356         // Remove nearby boxes of the same card (approx. same region)
00357         remainingGT.erase(
00358             std::remove_if(remainingGT.begin(), remainingGT.end(),
00359                 [&](const Detection& d) {
00360                     return (d.classId == matchedClass && calculateIoU(matchedRect, d.boundingBox)
00361 > 0.1f);
00362                 })),
00363             remainingGT.end()
00364         );
00365     } else {
00366         frameResults.fp++; //NOTE: No valid ground truths found for this prediction
00367     }
00368 }
00369
00370 // NOTE: Ricordati che marco ha etichettato il video in modo strano !
00371 // Remains ground truths are false negative
00372 frameResults.fn = static_cast<int>(remainingGT.size());
00373
00374 return frameResults;
00375 }
00376
00377 void printFinalMetrics(const MetricsCounter& total) {
00378     if (total.tp + total.fp == 0 || total.tp + total.fn == 0) {
00379         cout << "[METRICS] Warning: Not enough data to compute Precision/Recall." << endl;
00380         return;
00381     }
00382
00383     float precision = static_cast<float>(total.tp) / (total.tp + total.fp);
00384     float recall = static_cast<float>(total.tp) / (total.tp + total.fn);
00385     float f1Score = (precision + recall > 0) ? (2.0f * (precision * recall) / (precision + recall)) :
00386     0.0f;
00387
00388     cout << "\n--- Final Performance Metrics ---" << endl;
00389     cout << "Total True Positives (TP): " << total.tp << endl;
00390     cout << "Total False Positives (FP): " << total.fp << endl;
00391     cout << "Total False Negatives (FN): " << total.fn << endl;
00392     cout << "-----" << endl;
00393     cout << "Precision: " << precision << endl;
00394     cout << "Recall: " << recall << endl;
00395     cout << "F1-Score: " << f1Score << endl;
00396     cout << "-----\n" << endl;
00397 }
00398
00399 std::vector<Detection> parseYoloLabels(string labelPath, int imgW, int imgH) {
00400     std::vector<Detection> actualDetections;
00401     ifstream file(labelPath);
00402     string line;
00403
00404     try {
00405         while (getline(file, line)) {
00406             if (line.empty()) continue;
00407
00408             stringstream ss(line);
00409             int classId;
00410             float x_center, y_center, w, h;
00411
00412             if (!(ss >> classId >> x_center >> y_center >> w >> h)) {
00413                 throw std::runtime_error("Invalid YOLO format in file: " + labelPath);
00414             }
00415
00416             //NOTE: Converting bounding box width and height from percentage to actual values
00417             int pixelW = static_cast<int>(w * imgW);
00418             int pixelH = static_cast<int>(h * imgH);
00419             //NOTE: Moving the bounding box center to the left top corner.
00420             int pixelX = static_cast<int>((x_center * imgW) - (pixelW / 2.0f));
00421             int pixelY = static_cast<int>((y_center * imgH) - (pixelH / 2.0f));
00422
00423             Detection gt;
00424             gt.classId = classId;
00425             gt.boundingBox = cv::Rect(pixelX, pixelY, pixelW, pixelH);
00426             gt.classConfidence = 1.0f;

```

```

00427         actualDetections.push_back(gt);
00428     }
00429     } catch (const std::exception& e) {
00430         cerr << "[ERROR] Reading image ground truth: " << e.what() << endl;
00431         file.close();
00432         return {}; // Return empty vector on error
00433     }
00434     file.close();
00435     return actualDetections;
00436 }
00437
00438 void computeImageMetrics(string imagePath, std::vector<Detection> predictedDetections) {
00439     if (!canComputeImageMetrics(imagePath)) {
00440         return;
00441     }
00442
00443     fs::path p(imagePath);
00444     string filename = p.stem().string();
00445     string labelPath = "../data/test/ground_truths/images/" + filename + ".txt";
00446
00447     cv::Mat img = cv::imread(imagePath);
00448     if (img.empty()) {
00449         cerr << "[ERROR] Could not open image to get dimensions: " << imagePath << endl;
00450         return;
00451     }
00452
00453     // Extraction of ground truth using the helper method
00454     std::vector<Detection> actualDetections = parseYoloLabels(labelPath, img.cols, img.rows);
00455
00456     if (!actualDetections.empty()) {
00457         MetricsCounter results = updateMetricsCounters(predictedDetections, actualDetections);
00458         printFinalMetrics(results);
00459     }
00460 }

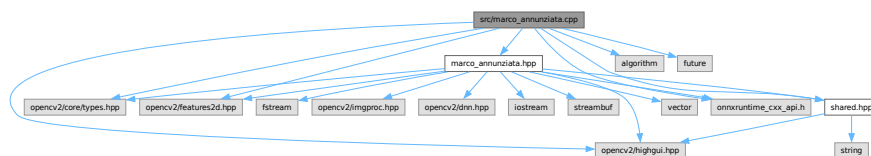
```

6.13 src/marco_annunziata.cpp File Reference

```

#include "marco_annunziata.hpp"
#include "shared.hpp"
#include <opencv2/core/types.hpp>
#include <opencv2/features2d.hpp>
#include <opencv2/highgui.hpp>
#include <onnxruntime_cxx_api.h>
#include <algorithm>
#include <future>
Include dependency graph for marco_annunziata.cpp:

```



Macros

- #define `UNUSED(x)`

6.13.1 Macro Definition Documentation

6.13.1.1 UNUSED

```
#define UNUSED(
    x)
```

Value:

```
(void) (x)
```

Definition at line 12 of file [marco_annunziata.cpp](#).

6.14 marco_annunziata.cpp

[Go to the documentation of this file.](#)

```
00001 // AUTHOR: Marco Annunziata. https://github.com/Remdoox
00002 #include "marco_annunziata.hpp"
00003 #include "shared.hpp"
00004 #include <opencv2/core/types.hpp>
00005 #include <opencv2/features2d.hpp>
00006 #include <opencv2/highgui.hpp>
00007 #include <onnxruntime_cxx_api.h>
00008 #include <algorithm>
00009 #include <future>
00010
00011 // A couple of variables could be useful to keep so I use this in order to avoid g++ warnings
00012 #define UNUSED(x) (void) (x)
00013
00014 using namespace std;
00015 using namespace cv;
00016 using namespace cv::dnn;
00017
00027 YOLO_model::YOLO_model(): env(ORT_LOGGING_LEVEL_VERBOSE, "YOLOModel", YOLO_model::logger, this),
    session(nullptr), sessionOptions() {
00028     if(usingGPU){
00029         try{
00030             OrtCUDAProviderOptions cudaOptions;
00031             cudaOptions.device_id = 0;
00032             cudaOptions.gpu_mem_limit = SIZE_MAX;
00033             // arena_extend_strategy: how much space to allocate at a time (arena = allocated block)
00034             // = 0 -> kNextPowerOfTwo, if the arena is filled up double its size (fast but
RAM-intensive)
00035             // = 1 -> kSameAsRequested, if the arena is filled up increase size exactly to what is
needed for the operation
00036             cudaOptions.arena_extend_strategy = 0;
00037             sessionOptions.AppendExecutionProvider_CUDA(cudaOptions);
00038             std::cout << "Using ONNX Runtime with GPU resources" << std::endl;
00039         }
00040         catch(const std::exception &e){
00041             std::cout << "CUDA not available! Using ONNX Runtime with CPU resources" << std::endl;
00042         }
00043     }
00044     unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00045     sessionOptions.SetIntraOpNumThreads(maxHardwareThreads);
00046     sessionOptions.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_EXTENDED);
00047     sessionOptions.SetLogId("YOLOModel");
00048
00049     const char* modelPath = "../data/model/YOLO11s_big_best_dynamic.onnx"; // YOLO11s_big_best_dynamic
00050     session = Ort::Session(env, modelPath, sessionOptions);
00051 }
00052
00053
00064 void YOLO_model::logger(void* param, OrtLoggingLevel severity, const char* category, const char*
logid, const char* code_location,
00065     const char* message){
00066     static std::mutex logMutex;
00067     static std::ofstream logFile("../onnxruntime.log", std::ios::out | std::ios::app);
00068
00069     const char* severityStr = "";
00070     switch (severity) {
00071         case ORT_LOGGING_LEVEL_VERBOSE: severityStr = "VERBOSE"; break;
00072         case ORT_LOGGING_LEVEL_INFO: severityStr = "INFO"; break;
00073         case ORT_LOGGING_LEVEL_WARNING: severityStr = "WARNING"; break;
00074         case ORT_LOGGING_LEVEL_ERROR: severityStr = "ERROR"; break;
```

```

00075         case ORT_LOGGING_LEVEL_FATAL: severityStr = "FATAL"; break;
00076         default: severityStr = "UNKNOWN"; break;
00077     }
00078
00079     // concatenating the output into a single string buffer to write into the log file
00080     std::ostringstream os;
00081     os << "[" << (logid ? logid : "no-logid") << "]" << " "
00082         << severityStr << " | " << (category ? category : "") << " @ "
00083         << (code_location ? code_location : "") << "\n" << " "
00084         << (message ? message : "") << "\n";
00085
00086     std::lock_guard<std::mutex> lock(logMutex);
00087
00088     if (severity <= ORT_LOGGING_LEVEL_INFO) {
00089         // INFO and VERBOSE go to file
00090         if (logFile) {
00091             logFile << os.str();
00092             logFile.flush();
00093         }
00094     } else {
00095         // WARNING, ERROR, FATAL go to stderr
00096         std::cerr << os.str();
00097     }
00098 }
00099
00104 bool YOLO_model::isAvailableGPU(){
00105     // Ottieni la lista dei provider compilati nel tuo ONNX Runtime
00106     std::vector<std::string> providers = Ort::GetAvailableProviders();
00107
00108     for (const auto& p : providers) {
00109         if (p == "CUDAXecutionProvider" || p == "DmlExecutionProvider") {
00110             this->usingGPU = true;
00111             return true;
00112         }
00113     }
00114     this->usingGPU = false;
00115     return false;
00116 }
00117
00118
00134 std::vector<Detection> YOLO_model::detectionPipeline(Mat &img, bool enable_letterbox_padding){
00135     int slice_size = 640;
00136     float overlap_ratio = 0.2;
00137     int step = slice_size * (1 - overlap_ratio);
00138
00139     // removes previous detections, in case of multiple subsequent detections
00140     clearDetections();
00141
00142     this->isAvailableGPU();
00143
00144     if(img.cols > 1.5*slice_size || img.rows > 1.5*slice_size){
00145         Detections mergedDetections;
00146         /* Tiled approach if image is too big (makes cards easier to detect for the model) */
00147         if(this->usingGPU){
00148             Detections tileDetections;
00149             for(int y = 0; y <= img.rows - slice_size; y += step){
00150                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00151                     cv::Rect roi(x, y, slice_size, slice_size);
00152                     cv::Mat tile = img(roi);
00153                     tileDetections = this->detect(tile, enable_letterbox_padding);
00154                     /* proceeding to map the box coordinates found in the tile to the whole image,
00155                     * then adding the detections of the specific tile to the detections of the overall
00156                     image
00157                     */
00158                     for(auto &box : tileDetections.boundingBoxes){
00159                         box.x += x;
00160                         box.y += y;
00161                     }
00162                     mergeDetections(mergedDetections, tileDetections);
00163                 }
00164             }
00165             /* Detection on the whole image improves result for large images having really large cards
00166             */
00167             mergeDetections(mergedDetections, this->detect(img, enable_letterbox_padding));
00168             this->detections = filterDetectionsNMS(mergedDetections);
00169         }
00170         else{
00171             unsigned int maxHardwareThreads = std::thread::hardware_concurrency();
00172             auto activeThreads = std::make_shared<std::atomic<int>>(0);
00173             std::vector<std::future<Detections>> futures;
00174
00175             for(int y = 0; y <= img.rows - slice_size; y += step){
00176                 for(int x = 0; x <= img.cols - slice_size; x+= step){
00177                     while((*activeThreads) >= (int) maxHardwareThreads){
00178                         std::this_thread::yield();
00179                     }

```

```

00179             (*activeThreads)++;
00180             cv::Rect roi(x, y, slice_size, slice_size);
00181             cv::Mat tile = img(roi).clone();
00182
00183             futures.push_back(std::async(std::launch::async, [this, tile,
enable_letterbox_padding, x, y, activeThreads]() {
00184                 Detections tileDetections = this->detect(tile, enable_letterbox_padding);
00185                 /* proceeding to map the box coordinates found in the tile to the whole image,
00186                  * then adding the detections of the specific tile to the detections of the
overall image */
00187                 for(auto &box : tileDetections.boundingBoxes){
00188                     box.x += x;
00189                     box.y += y;
00190                 }
00191                 (*activeThreads)--;
00192                 return tileDetections;
00193             }));
00194         }
00195     }
00196     /* Detect on the whole image, collect detections and merge them, then filter with NMS
suppression */
00197     futures.push_back(std::async(std::launch::async, [this, img, enable_letterbox_padding]() {
return this->detect(img, enable_letterbox_padding); }));
00198     for(auto& future : futures) mergeDetections(mergedDetections, future.get());
00199     this->detections = filterDetectionsNMS(mergedDetections);
00200 }
00201 }
00202 else this->detections = filterDetectionsNMS(this->detect(img, enable_letterbox_padding));
00203 return this->detections;
00204 }
00205
00207 void YOLO_model::clearDetections() {
00208     this->detections.clear();
00209 }
00210
00218 void YOLO_model::mergeDetections(Detections& dest, const Detections& source) {
00219     if(source.boundingBoxes.empty()) return;
00220
00221     size_t newSize = dest.boundingBoxes.size() + source.boundingBoxes.size();
00222
00223     dest.boundingBoxes.reserve(newSize);
00224     dest.classConfidences.reserve(newSize);
00225     dest.classIds.reserve(newSize);
00226
00227     dest.boundingBoxes.insert(dest.boundingBoxes.end(), source.boundingBoxes.begin(),
source.boundingBoxes.end());
00228     dest.classConfidences.insert(dest.classConfidences.end(), source.classConfidences.begin(),
source.classConfidences.end());
00229     dest.classIds.insert(dest.classIds.end(), source.classIds.begin(), source.classIds.end());
00230 }
00231 }
00232
00243 Detections YOLO_model::detect(const Mat &img, bool enable_letterbox_padding) {
00244     /* Note: there is small to no documentation of the C++ implementation of OnnxRuntime even from the
developers themselves, so this function has been thoroughly optimized and documented
00245     such that every step is perfectly clear to everyone reading this code. */
00246
00247     int inputHeight = img.rows;
00248     int inputWidth = img.cols;
00249
00250     /* It's possible to give an image as input without resizing in OnnxRuntime because it supports
dynamic input, but the processing of the model is way slower.
00251     The input needs to be resized anyway for the YOLO model to have good accuracy, so
pre-processing of the image is still needed and the dynamic input feature is not used for YOLO */
00252
00253     /* YOLO: PRE-PROCESSING required for the image
the YOLO model requires the image to be:
00254     1. Of dimensions multiples of 32, because we know YOLO has a CNN in its architecture:
-There are 5 convolutional layers with stride=2 where the convolutions downsample
the input by a factor of 2^5=32, so the feature map has size inputSize/32
00257     -The stride of convolution replaces max pooling of typical CNNs
00258     -If the inputSize is not divisible by 32, the feature map become of size
non-integer which leads to feature misalignment
00259     2. Of overall size square (multiple of 32), typically 640x640 or 416x416
00260     3. not deformed (deformed = stretched image), since YOLO doesn't recognize deformed
objects and, even if you train it for that, the predictions are still made in fixed grid cells
00261     This means that:
00262     - If an image is smaller or bigger than 640x640, it has to be resized to the same scale of
640x640 (requirement 2)
00263     - If the image is not square, you need to resize it while keeping its aspect ratio w:h, so
that the result is not deformed (requirement 3)
00264     => the scaling factor has to be saved so that you can correctly resize the bounding box
to the original size of the image
00265     - If the image is not square, after resizing you also need to add padding to have a
resulting image of exactly 640x640 size (requirement 2)
00266     So the pre-processing of the images is composed of these steps:
00267     Case A: If the image is square, just resize it if needed and you're done

```

```

00268         Case B: If the image is not square (most of the time):
00269             1. Resize the image to a multiple of 32 while keeping its aspect ratio, which means:
resize the longest side to 640 and adjust the shorter side following proportions w:h
00270             2. add padding to the shorter side in order to reach the square size.
00271         Note: to guarantee the best performance, the padding has to be done in the same we YOLO was
trained for.
00272         YOLO was trained using images with letterbox padding, which means that the image is centered
and the padding is around the image.
00273         This is useful because in the uncentered padding the model would have a small bias of
detecting objects which are closer to the corner.
00274         A symmetric padding (letterbox) is more robust to this positional bias.
00275     */
00276
00277     Mat resizedImg;
00278     float resizedWidth    = -1;
00279     float resizedHeight   = -1;
00280     int xPaddedImgCorner  = 0; // It represents the upper left corner of the centered image after
letterbox padding, will be useful later to center the image
00281     int yPaddedImgCorner  = 0;
00282
00283     float imgAspectRatio  = static_cast<float>(inputWidth) / inputHeight;
00284     if(inputWidth == inputHeight){
00285         /* Case A: image is square, just resize */
00286         resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00287         resizedHeight = YOLO_TARGET_INPUT_SIZE;
00288         resize(img, resizedImg, Size(static_cast<int>(resizedWidth),
static_cast<int>(resizedHeight)));
00289     }
00290     else{
00291         /* Case B: image is not square. Resize following aspect ratio, then add letterbox padding*/
00292         if(imgAspectRatio > 1.0f){
00293             resizedWidth  = YOLO_TARGET_INPUT_SIZE;
00294             resizedHeight = YOLO_TARGET_INPUT_SIZE / imgAspectRatio; // imgAspectRatio > 1 -> have to
divide so that the height follows the width increase/decrease
00295             resizedHeight = (static_cast<int>(resizedHeight) / 32) * 32; // Ensure dimensions of the
image are multiples of 32, as required by YOLO11 (in this case width is already multiple of 32)
00296
00297             if(enable_letterbox_padding == true) yPaddedImgCorner =
static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedHeight/2);
00298         }
00299         else{
00300             resizedHeight = YOLO_TARGET_INPUT_SIZE;
00301             resizedWidth  = YOLO_TARGET_INPUT_SIZE * imgAspectRatio; // imgAspectRatio < 1 -> have to
multiply to make the height follow the width value
00302             resizedWidth  = (static_cast<int>(resizedWidth) / 32) * 32;
00303
00304             if(enable_letterbox_padding == true) xPaddedImgCorner =
static_cast<int>((YOLO_TARGET_INPUT_SIZE/2) - resizedWidth/2);
00305         }
00306
00307         Mat tempImg;
00308         resize(img, tempImg, Size(static_cast<int>(resizedWidth), static_cast<int>(resizedHeight)));
00309
00310         //letterbox padding
00311         if(enable_letterbox_padding == true){
00312             resizedImg = Mat::zeros(YOLO_TARGET_INPUT_SIZE, YOLO_TARGET_INPUT_SIZE, CV_8UC3);
00313             tempImg.copyTo(resizedImg(Rect(xPaddedImgCorner, yPaddedImgCorner, resizedWidth,
resizedHeight)));
00314         }
00315         else resizedImg = tempImg.clone();
00316     }
00317
00318     float resizeScalingFactor[2] = {
00319         static_cast<float>(inputWidth) / resizedWidth,
00320         static_cast<float>(inputHeight) / resizedHeight
00321     };
00322
00323     /* TENSORS
00324     OnnxRuntime needs a tensor as input.
00325     The ONNX tensor is not the mathematical tensor (although it is inspired by it).
00326     It's a multi-dimensional array of heterogeneous data types (float, int ...) which is optimized for
computations.
00327     Tensors are useful because they are a convenient data structure for parallelizing GPU computations
during DNN inference.*/
00328
00329     /* INPUT TENSOR
00330     * In OnnxRuntime the tensor is defined by its data (an array) and its shape (dimensions of the
multi-dimensional array)
00331     * Note: ONNX has the convention to have the nput tensor shape following the NCHW order, where N =
batch size, C = n°channels, H = height, W = width of the image.
00332     * For convention height comes before width because data in memory is accessed along the row:
data[y][x]
00333     * You can check the shape of input tensor of your model by inspecting the .onnx file with
https://netron.app/
00334     */
00335
00336     /* Preparing the parameters for the dynamic INPUT TENSOR */

```



```

00337     vector<int64_t> inputShape = {1, 3, static_cast<int>(resizedHeight),
static_cast<int>(resizedWidth)}; // 4D tensor
00338     size_t inputTensorSize = 1 * 3 * resizedHeight * resizedWidth;
00339     vector<float> inputData(inputTensorSize); // the input image is flattened into a mono-dimensional
array
00340
00341     // Converting image data from cv::Mat to tensor (normalization to [0,1] and converting (cv::Mat)
BGR to RGB as the YOLO model requires)
00342     double normalizationFactor = 1.0 / 255.0;
00343     for (int row = 0; row < resizedHeight; row++) {
00344         for (int col = 0; col < resizedWidth; col++) {
00345             Vec3b pixel = resizedImg.at<Vec3b>(row, col);
// cv::Mat -> Tensor
00346             inputData[2 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[0] *
normalizationFactor; // B -> R
00347             inputData[1 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[1] *
normalizationFactor; // G -> G
00348             inputData[0 * resizedHeight * resizedWidth + row * resizedWidth + col] = pixel[2] *
normalizationFactor; // R -> B
00349         }
00350     }
00351
00352     auto memoryInfo = Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault); // Allocating
the input tensor on CPU
00353
00354     // Creating the input tensor (which is a view of the data, NOT a deep copy!)
00355     Ort::Value inputOnnxTensor = Ort::Value::CreateTensor<float>(memoryInfo, inputData.data(),
inputTensorSize, inputShape.data(), inputShape.size());
00356
00357     // retrieving now as string the name of the first input and output nodes
00358     // the allocations of these strings are automatically freed by the session
00359     auto inputNodeName = session.GetInputNameAllocated(0, allocator);
00360     auto outputNodeName = session.GetOutputNameAllocated(0, allocator);
00361
00362     // The API needs the array of inputs to set and the array of outputs to get (or even just a
portion of output if needed).
00363     // So here you are setting the ORDER of the input tensors and the ORDER of the outputs.
00364     // This step is useful for models which have multiple input and output tensors where the order
matters.
00365     // In our case we just have a 1 to 1 correspondence of input-output so the arrays passed will only
have 1 item
00366     vector<const char*> inputNodeNames = {inputNodeName.get()};
00367     vector<const char*> outputNodeNames = {outputNodeName.get()};
00368
00369     // Running the inference
00370     auto outputTensors = session.Run(
00371         Ort::RunOptions{nullptr}, // e.g set a verbosity level only for this run
00372         inputNodeNames.data(), &inputOnnxTensor, inputNodeNames.size(), // input to set (=1 input
image)
00373         outputNodeNames.data(), // output to take
00374         outputNodeNames.size() // actually = 1 output tensors because YOLO has only 1
output layer
00375     );
00376
00377     /* OUTPUT TENSOR
00378     // YOLO11 is pre-trained by default on the COCO dataset, so the number of channels should be:
00379     // 84 = 4 + 80 (= class confidence, one for each class)
00380     // BUT, we fine-tuned the training of YOLO11 with our custom dataset, so the output layer is
restructured to represent 53 classes.
00381     // This means the number of channels should be 57 = 4 (bounding box parameters) + 53 (= class
confidence, one for each class)
00382     // Note: class probability = probability of having detected one class in that region
00383     // class confidence = confidence of having detected an object of a class in that
region, measured as class probability * objectness probability
00384     // objectness probability = probability of having an object in that region, useful for
YOLOv5 but not for YOLO11, since there is already class confidence, so it's been removed.
00385
00386     // Note, YOLOv5 has a different representation of the output layer compared to YOLOv8/YOLO11:
00387     // YOLOv5 -> channels = 85 = 4 (bb_parameters) + 1 (objectness probability) + 80 (= class
probabilities of corresponding 80 classes of the COCO dataset)
00388     // So, in this case the objectness is separate from class confidence and each class has the class
probability instead of the class confidence.
00389     // To get class confidence of each class you have to multiply each class probability*objectness.
So in YOLOv5 the objectness probability was useful!
00390
00391     // To recap, our custom YOLO11 output tensor (3D Mat) has shape (1, 57, 8400*):
00392     // 1 - is the batch size. If =0, it can change dynamically depending
on the input of the net (not needed since in YOLO batch size = 1).
00393     // 57 (84 for default model) - is the number of channels, where the first 4 are the bounding
box parameters (x, y, width, height)
00394     // and the other ones are class probabilities for COCO'S 80
classes (There is no objectiveness parameter in YOLOv8/11)
00395     // 8400 *or lower - is the total number of regions where the predictions are made
("regions" = "grid points" = anchor locations).
00396     // *Since the ONNX model is dynamic, the output tensor can have
a lower number of detections depending on how big is the padding area.
00397     // The first and the third value become 0 when exported into ONNX format with dynamic=True,

```

```

    indicating that in the dynamic model these parameters can change on runtime.
00398 // Dynamic input/output shapes are not supported by OpenCV, but they are supported by ONNX
Runtime.
00399 // In this case, OnnxRuntime is adopted instead of OpenCV more because of the superior inference
efficiency rather than the support of dynamic models.
00400 // You can also see the shape of the ouput layer by importing the .onnx file in
https://netron.app/
00401 // See also: https://github.com/orgs/ultralytics/discussions/17254
00402 */
00403
00404 /* Getting dynamic output tensor info (data, shape, count) */
00405 auto& outputTensor = outputTensors[0]; // In the case of YOLO11, only 1 output layer is used, so
there is only 1 output tensor (outputTensors[0])
00406 auto tensorInfo = outputTensor.GetTensorTypeAndShapeInfo();
00407 auto tensorCount = tensorInfo.GetElementCount(); UNUSED(tensorCount);
00408 auto tensorShape = tensorInfo.GetShape();
00409 float* outputData = outputTensor.GetTensorMutableData<float>(); // the output values are flattened
into a 1D array
00410 // YOLO11: outputData[0..4...] = (x, y, width, height, confidence, classConfidence1,
classConfidence2, ...)
00411
00412 int batchSize = static_cast<int>(tensorShape[0]); UNUSED(batchSize);
00413 int numChannels = static_cast<int>(tensorShape[1]);
00414 int numDetections = static_cast<int>(tensorShape[2]); // The number of detections will vary based
on input size and padding area
00415
00416 // NOTE: The output tensor is a matrix like this, if the batch size =1: (neglecting the first four
parameters used for the bounding box)
00417 //      class1_detection1      class1_detection2      ...      class1_detectionN
00418 //      ....
00419 //      class57_detection1      class57_detectionN
00420
00421 Detections goodDetections;
00422 vector<int> classIds;
00423 vector<float> detectedClassConfidences;
00424 vector<Rect> boundingBoxes;
00425
00426 /* Declaring things used in the post-process phase */
00427 Mat classScores;
00428 Point classId;
00429 double maxClassScore;
00430
00431 // I want to iterate through the output as a flat 1D array, so that i can use a pointer instead of
copying values
00432 // When doing this, I need the output to have detections as rows and classes as columns so that I
can evaluate the bounding box of each detection.
00433 //      class1_detection1      class1_detection2      ...      class1_detectionN
TRANSPOSE      detection1_class1      detection1_class2      ...      detection1_class57
00434 //      ....
--->      ....
00435 //      class57_detection1      class57_detectionN
detectionN_class1      detectionN_class57
00436 // This is why a Mat is constructed from the pointer outputData and its transposition t() is taken
00437 Mat outputDataMat= Mat(numChannels, numDetections, CV_32F, outputData).t(); // -> now the data is
in the order we needed! [Det1Class1 Det1Class2 ... Det1Class57 Det2Class1 ...]
00438 // -> This Mat constructor with the fourth paramter is defined as:
00439 // Mat(int rows, int cols, int type, void* data, size_t step=AUTO_STEP);
00440 // The fourth parameter is just a pointer to the data, it doesn't need to copy the values!
00441 // So this constructor just wraps pointer outputData in a matrix header, resulting a lot more
efficient.
00442
00443 for(int i = 0; i < numDetections; i++){
00444     float* currentDetection = outputDataMat.ptr<float>(i); // 1 detection corresponds to a 1D
array of the 57 elements of the detection
00445     float* firstClassScore = currentDetection + 4;
00446     classScores = Mat(1, numChannels-4, CV_32FC1, firstClassScore); // using efficient constructor
00447     // Also, minMaxLoc accepts as input a Mat and as output a Point, so this is why classScores
and classId are defined as Mat and Point.
00448
00449     minMaxLoc(classScores, 0, &maxClassScore, 0, &classId);
00450
00451     if(maxClassScore > CLASS_CONFIDENCE_THRESHOLD){
00452         detectedClassConfidences.push_back(maxClassScore);
00453         classIds.push_back(classId.x);
00454
00455         float xCenter = currentDetection[0];
00456         float yCenter = currentDetection[1];
00457         float boxWidth = currentDetection[2];
00458         float boxHeight = currentDetection[3];
00459
00460         /* removing coordinates translation caused by the padding */
00461         if(enable_letterbox_padding == true){
00462             xCenter -= xPaddedImgCorner;
00463             yCenter -= yPaddedImgCorner;
00464         }
00465
00466         /* computing the bounding box of the detected object and scaling it to the original image

```

```

size */
00467         int xPosScaled = int((xCenter - 0.5f*boxWidth) * resizeScalingFactor[0]);
00468         int yPosScaled = int((yCenter - 0.5f*boxHeight) * resizeScalingFactor[1]);
00469         int bWScaled = int(boxWidth * resizeScalingFactor[0]);
00470         int bHScaled = int(boxHeight * resizeScalingFactor[1]);
00471
00472         Rect box = Rect(xPosScaled, yPosScaled, bWScaled, bHScaled);
00473
00474         /* Clamping bounding box to image boundaries */
00475         box.x = max(0, box.x);
00476         box.y = max(0, box.y);
00477         box.width = min(box.width, img.cols - box.x);
00478         box.height = min(box.height, img.rows - box.y);
00479
00480         boundingBoxes.push_back(box);
00481     }
00482 }
00483
00484 goodDetections.boundingBoxes = boundingBoxes;
00485 goodDetections.classConfidences = detectedClassConfidences;
00486 goodDetections.classIds = classIds;
00487
00488 return goodDetections;
00489 /* Non-Maxima Suppression and construction of the final array of detections */
00490 }
00491
00498 std::vector<Detection> YOLO_model::filterDetectionsNMS(Detections goodDetections){
00499     vector<int> resultNMS;
00500     cv::dnn::NMSBoxes(goodDetections.boundingBoxes, goodDetections.classConfidences,
00501         CLASS_CONFIDENCE_THRESHOLD, NMS_THRESHOLD, resultNMS);
00502     std::vector<Detection> finalDetections;
00503
00504     for(auto finalBoxIndex : resultNMS){
00505         Detection finalDetection;
00506         finalDetection.classId = goodDetections.classIds[finalBoxIndex];
00507         finalDetection.classConfidence = goodDetections.classConfidences[finalBoxIndex];
00508         finalDetection.boundingBox = goodDetections.boundingBoxes[finalBoxIndex];
00509         finalDetections.push_back(finalDetection);
00510     }
00511     return finalDetections;
00512 }
00513
00519 vector<string> YOLO_model::getDataClasses(string labelsFilename){
00520     vector<string> dataClasses;
00521     ifstream ifs(labelsFilename);
00522     string line;
00523     while (getline(ifs, line))
00524         dataClasses.push_back(line);
00525     this->classNames = dataClasses;
00526     return dataClasses;
00527 }
00528
00537 vector<string> YOLO_model::getDetectionsClassNames(Detections detections){
00538     std::vector<string> detectionsClassNames;
00539     for(int id: detections.classIds){
00540         detectionsClassNames.push_back(this->classNames[id]);
00541     }
00542     return detectionsClassNames;
00543 }
00544
00552 std::string YOLO_model::getClassName(int classId){
00553     return this->classNames[classId];
00554 }
00555
00565 cv::Mat YOLO_model::drawBoundingBoxes(cv::Mat &img, std::vector<Detection> &detections, cv::Scalar
color){
00566     Mat resultImg = img.clone();
00567     for (auto detection : detections)
00568     {
00569         int thickness = max(1, int(max(img.rows, img.cols) / 640));
00570         rectangle(resultImg, detection.boundingBox, color, 2 * thickness);
00571         string label = this->getClassName(detection.classId) + " " +
00572             to_string(static_cast<int>(detection.classConfidence * 100)) + "%";
00573         putText(resultImg, label, Point(detection.boundingBox.x, detection.boundingBox.y - 5 *
thickness), FONT_HERSHEY_SIMPLEX, 0.5 * thickness, color, 1.5 * thickness);
00574     }
00575     // std::string windowTitle = getModelName() + " - " + std::to_string(detections.size()) + "
detections";
00576     // namedWindow(windowTitle, WINDOW_NORMAL);
00577     // imshow(windowTitle, resultImg);
00578     // waitKey(0);
00579     return resultImg;
00580 }
00581
00589 cv::Mat YOLO_model::drawBoundingBoxes(cv::Mat &img, cv::Scalar color){
00590     return drawBoundingBoxes(img, this->detections, color);
00591 }

```

```

00592
00596 void YOLO_model::setModelName(string modelName){
00597     this->modelName = modelName;
00598 }
00599
00603 string YOLO_model::getModelName(){
00604     return this->modelName;
00605 }
00606
00610 std::vector<Detection> YOLO_model::getDetections(){
00611     return this->detections;
00612 }

```

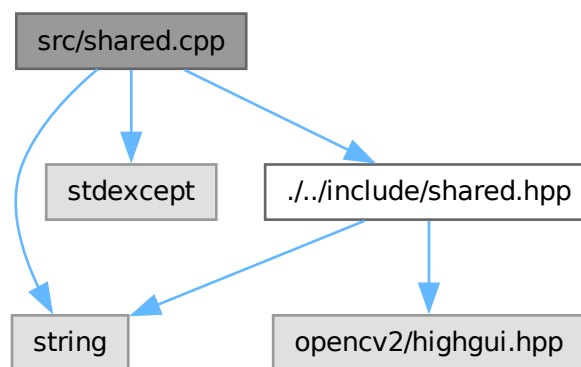
6.15 src/shared.cpp File Reference

```

#include <string>
#include <stdexcept>
#include "../include/shared.hpp"

```

Include dependency graph for shared.cpp:



Namespaces

- namespace [Shared](#)

Functions

- [Choice Shared::parseChoice](#) (std::string s)

6.16 shared.cpp

[Go to the documentation of this file.](#)

```

00001 #include <string>
00002 #include <stdexcept>
00003
00004 #include "../include/shared.hpp"
00005

```



```
int savedCount,
string labels_path)
```

Definition at line 19 of file [sveva_turola.cpp](#).

```
00019 {
00020     YOLO_model model;
00021     model.setModelName("YOLO11s");
00022     vector<string> dataClasses = model.getDataClasses(labels_path);
00023     int frameCount = 0;
00024     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00025     int fps = cap.get(CAP_PROP_FPS);
00026     vector<Detection> detections;
00027
00028     cap.read(frame);
00029     if (frame.empty()){
00030         return -1;
00031     }
00032
00033     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00034     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00035     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00036     if(!out.isOpened()){
00037         cerr << "Could not open the output video file for write\n";
00038         return -1;
00039     }
00040
00041     cout << "Loading camera video..." << endl;
00042
00043     while (true) {
00044         cap.read(frame);
00045         if (frame.empty()){
00046             break;
00047         }
00048
00049         savedCount++;
00050         int key = waitKey(1) & 0xFF;
00051
00052         if (frameCount % (fps/2) == 0) { // TODO cambiare
00053             detections = model.detectionPipeline(frame);
00054         }
00055         frameCount++;
00056         Mat outputFrame = cardValues(detections, model, frame);
00057         out.write(outputFrame);
00058         namedWindow("Output Video", WINDOW_NORMAL);
00059         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00060         imshow("Output Video", outputFrame);
00061         waitKey(1);
00062
00063         if (key == 'q') {
00064             cout << "Closed camera!\n";
00065             break;
00066         }
00067     }
00068
00069     return savedCount;
00070 }
```

6.17.1.2 cardValues()

```
Mat cardValues (
    vector< Detection > detections,
    YOLO_model & model,
    Mat & frame)
```

Definition at line 155 of file [sveva_turola.cpp](#).

```
00155 {
00156     vector<Detection> green, blue, red;
00157
00158     for(auto detection : detections){
00159         char cardNumber = model.getClassName(detection.classId)[0];
00160
00161         if(cardNumber >= '2' && cardNumber <= '6'){
00162             green.push_back(detection);
00163         }
00164         else if(cardNumber >= '7' && cardNumber <= '9'){
00165             blue.push_back(detection);
00166         }
00167         else {
```

```

00168         red.push_back(detection);
00169     }
00170 }
00171
00172 Mat outputFrame = frame.clone();
00173
00174 if(!green.empty()){
00175     outputFrame = model.drawBoundingBoxes(outputFrame, green, Scalar(0, 255, 0));
00176 }
00177 if(!blue.empty()){
00178     outputFrame = model.drawBoundingBoxes(outputFrame, blue, Scalar(255, 0, 0));
00179 }
00180 if(!red.empty()){
00181     outputFrame = model.drawBoundingBoxes(outputFrame, red, Scalar(0, 0, 255));
00182 }
00183
00184 return outputFrame;
00185 }

```

6.17.1.3 frameCapture()

```

int frameCapture (
    string data_path,
    string labels_path)

```

Definition at line 119 of file [sveva_turola.cpp](#).

```

00119 {
00120     string outputDir = "../output/frames/";    //output directory
00121     Mat frame;
00122     int savedCount = 0;
00123     VideoCapture cap;
00124     VideoWriter out;
00125
00126     if (!fs::exists(outputDir)) { // creation of output directory
00127         fs::create_directory(outputDir);
00128     }
00129
00130     if(data_path == "0"){
00131         cap.open(0);
00132         if (!cap.isOpened()) {
00133             cerr << "Error opening camera!" << endl;
00134             return -1;
00135         }
00136         savedCount = cameraCapture(cap, out, frame, savedCount, labels_path);
00137     } else {
00138         cap.open(data_path);
00139         if (!cap.isOpened()) {
00140             cerr << "Error opening video!" << endl;
00141             return -1;
00142         }
00143         savedCount = videoCapture(cap, out, frame, savedCount, labels_path);
00144     }
00145
00146     cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00147     cout << "Video saved in output directory" << endl;
00148
00149     cap.release();
00150     out.release();
00151     destroyAllWindows();
00152     return 0;
00153 }

```

6.17.1.4 videoCapture()

```

int videoCapture (
    VideoCapture cap,
    VideoWriter out,
    Mat frame,
    int savedCount,
    string labels_path)

```

Definition at line 72 of file [sveva_turola.cpp](#).

```

00072                                     {
00073     YOLO_model model;
00074     model.setModelName("YOLO11s");
00075     vector<string> dataClasses = model.getDataClasses(labels_path);
00076     int frameCount = 0;
00077     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00078     int fps = cap.get(CAP_PROP_FPS);
00079     vector<Detection> detections;
00080
00081     cap.read(frame);
00082     if (frame.empty()){
00083         return -1;
00084     }
00085
00086     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00087     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00088     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00089     if(!out.isOpened()){
00090         cerr << "Could not open the output video file for write\n";
00091         return -1;
00092     }
00093
00094     cout << "Loading video..." << endl;
00095
00096     while (true) {
00097         cap.read(frame);
00098         if (frame.empty()){
00099             break;
00100         }
00101
00102         savedCount++;
00103         int key = waitKey(1) & 0xFF;
00104
00105         if (frameCount % (fps/2) == 0) { //TODO cambiare
00106             detections = model.detectionPipeline(frame);
00107         }
00108         frameCount++;
00109         Mat outputFrame = cardValues(detections, model, frame);
00110         out.write(outputFrame);
00111         namedWindow("Output Video", WND_PROP_FULLSCREEN);
00112         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00113         imshow("Output Video", outputFrame);
00114         waitKey(1);
00115     }
00116     return savedCount;
00117 }

```

6.18 sveva_turola.cpp

[Go to the documentation of this file.](#)

```

00001 #include <opencv2/features2d.hpp>
00002 #include <opencv2/highgui.hpp>
00003 #include <iostream>
00004 #include <opencv2/core.hpp>
00005 #include <opencv2/imgproc.hpp>
00006 #include <opencv2/calib3d.hpp>
00007 #include <filesystem>
00008 #include <opencv2/videoio.hpp>
00009 #include <string>
00010 #include <vector>
00011
00012 #include "../include/sveva_turola.hpp"
00013 #include "../include/marco_annunziata.hpp"
00014
00015 using namespace std;
00016 using namespace cv;
00017 namespace fs = std::filesystem;
00018
00019 int cameraCapture(VideoCapture cap, VideoWriter out, Mat frame, int savedCount, string labels_path){
00020     YOLO_model model;
00021     model.setModelName("YOLO11s");
00022     vector<string> dataClasses = model.getDataClasses(labels_path);
00023     int frameCount = 0;
00024     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00025     int fps = cap.get(CAP_PROP_FPS);
00026     vector<Detection> detections;
00027
00028     cap.read(frame);
00029     if (frame.empty()){
00030         return -1;
00031     }

```



```

00032
00033     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00034     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00035     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00036     if(!out.isOpened()){
00037         cerr << "Could not open the output video file for write\n";
00038         return -1;
00039     }
00040
00041     cout << "Loading camera video..." << endl;
00042
00043     while (true) {
00044         cap.read(frame);
00045         if (frame.empty()){
00046             break;
00047         }
00048
00049         savedCount++;
00050         int key = waitKey(1) & 0xFF;
00051
00052         if (frameCount % (fps/2) == 0) { // TODO cambiare
00053             detections = model.detectionPipeline(frame);
00054         }
00055         frameCount++;
00056         Mat outputFrame = cardValues(detections, model, frame);
00057         out.write(outputFrame);
00058         namedWindow("Output Video", WINDOW_NORMAL);
00059         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00060         imshow("Output Video", outputFrame);
00061         waitKey(1);
00062
00063         if (key == 'q') {
00064             cout << "Closed camera!\n";
00065             break;
00066         }
00067     }
00068
00069     return savedCount;
00070 }
00071
00072 int videoCapture.VideoCapture cap, VideoWriter out, Mat frame, int savedCount, string labels_path){
00073     YOLO_model model;
00074     model.setModelName("YOLO11s");
00075     vector<string> dataClasses = model.getDataClasses(labels_path);
00076     int frameCount = 0;
00077     int codec = VideoWriter::fourcc('m', 'p', '4', 'v');
00078     int fps = cap.get(CAP_PROP_FPS);
00079     vector<Detection> detections;
00080
00081     cap.read(frame);
00082     if (frame.empty()){
00083         return -1;
00084     }
00085
00086     int frame_width = static_cast<int>(cap.get(CAP_PROP_FRAME_WIDTH));
00087     int frame_height = static_cast<int>(cap.get(CAP_PROP_FRAME_HEIGHT));
00088     out.open("../output/detections.mp4", codec, fps, Size(frame_width, frame_height), true);
00089     if(!out.isOpened()){
00090         cerr << "Could not open the output video file for write\n";
00091         return -1;
00092     }
00093
00094     cout << "Loading video..." << endl;
00095
00096     while (true) {
00097         cap.read(frame);
00098         if (frame.empty()){
00099             break;
00100         }
00101
00102         savedCount++;
00103         int key = waitKey(1) & 0xFF;
00104
00105         if (frameCount % (fps/2) == 0) { //TODO cambiare
00106             detections = model.detectionPipeline(frame);
00107         }
00108         frameCount++;
00109         Mat outputFrame = cardValues(detections, model, frame);
00110         out.write(outputFrame);
00111         namedWindow("Output Video", WND_PROP_FULLSCREEN);
00112         setWindowProperty("Output Video", WND_PROP_FULLSCREEN, WINDOW_FULLSCREEN);
00113         imshow("Output Video", outputFrame);
00114         waitKey(1);
00115     }
00116     return savedCount;
00117 }
00118

```

```

00119 int frameCapture(string data_path, string labels_path) {
00120     string outputDir = "../output/frames/"; //output directory
00121     Mat frame;
00122     int savedCount = 0;
00123     VideoCapture cap;
00124     VideoWriter out;
00125
00126     if (!fs::exists(outputDir)) { // creation of output directory
00127         fs::create_directory(outputDir);
00128     }
00129
00130     if(data_path == "0"){
00131         cap.open(0);
00132         if (!cap.isOpened()) {
00133             cerr << "Error opening camera!" << endl;
00134             return -1;
00135         }
00136         savedCount = cameraCapture(cap, out, frame, savedCount, labels_path);
00137     } else {
00138         cap.open(data_path);
00139         if (!cap.isOpened()) {
00140             cerr << "Error opening video!" << endl;
00141             return -1;
00142         }
00143         savedCount = videoCapture(cap, out, frame, savedCount, labels_path);
00144     }
00145
00146     cout << "\nExtraction completed! Frames used: " << savedCount << endl;
00147     cout << "Video saved in output directory" << endl;
00148
00149     cap.release();
00150     out.release();
00151     destroyAllWindows();
00152     return 0;
00153 }
00154
00155 Mat cardValues(vector<Detection> detections, YOLO_model &model, Mat &frame){
00156     vector<Detection> green, blue, red;
00157
00158     for(auto detection : detections){
00159         char cardNumber = model.getClassName(detection.classId)[0];
00160
00161         if(cardNumber >= '2' && cardNumber <= '6'){
00162             green.push_back(detection);
00163         }
00164         else if(cardNumber >= '7' && cardNumber <= '9'){
00165             blue.push_back(detection);
00166         }
00167         else {
00168             red.push_back(detection);
00169         }
00170     }
00171
00172     Mat outputFrame = frame.clone();
00173
00174     if(!green.empty()){
00175         outputFrame = model.drawBoundingBoxes(outputFrame, green, Scalar(0, 255, 0));
00176     }
00177     if(!blue.empty()){
00178         outputFrame = model.drawBoundingBoxes(outputFrame, blue, Scalar(255, 0, 0));
00179     }
00180     if(!red.empty()){
00181         outputFrame = model.drawBoundingBoxes(outputFrame, red, Scalar(0, 0, 255));
00182     }
00183
00184     return outputFrame;
00185 }

```